

SOLIDWORKS API (.NET VBA C#) — Examples (23/50):

Get_and_Set_Seed_Components_Example_VBNE- Get_Arcs_in_Sketch_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get and Set Seed Components Example (VB.NET)

This example shows how to access the seed components of patterns in an assembly. It also shows that seed components can be replaced by either IComponent2 objects or IFeature objects representing those components.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate Window.
'
' Postconditions:
' 1. Gets the names of the seed components.
' 2. Replaces the seed components by an IComponent2 object
'    or an IFeature object representing the component.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim dispWrapArr() As DispatchWrapper

        Dim fileName As String
        Dim errors As Long
        Dim warnings As Long
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"

        ' Open the model
        Dim swModel As ModelDoc2
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)

        ' Verify that the model is active
        If swModel Is Nothing Then
            Debug.Print("No active model document")
            Exit Sub
        End If

        Debug.Print("Model: " & swModel.GetTitle)

        ' Get all of the features in the model
        Dim swFeatMgr As FeatureManager
        swFeatMgr = swModel.FeatureManager

        Dim vFeatures As Object
        vFeatures = swFeatMgr.GetFeatures(True)

        ' Iterate over all features
        Dim iFeat As Integer
        For iFeat = LBound(vFeatures) To UBound(vFeatures)
            Dim swFeature As Feature
            swFeature = vFeatures(iFeat)

            ' Is the current feature a patterned feature?
            Select Case UCase(swFeature.GetTypeName2)
                Case "LOCALLPATTERN"
                    ' ILocalLinearPatternFeatureData
                    Debug.Print("    Linear Pattern: " & swFeature.Name)
                    Dim llpDefinition As LocalLinearPatternFeatureData
                    llpDefinition = swFeature.GetDefinition
                    Dim llpSeedComps() As Object
                    ReDim llpSeedComps(0)
                    If llpDefinition.AccessSelections(swModel, Nothing) Then
                        ' Get the seed components for this pattern
                        llpSeedComps = llpDefinition.SeedComponentArray
                        ReDim Preserve llpSeedComps(UBound(llpSeedComps))
                    End If

                    ' Arbitrarily decide whether to replace each seed component
                    ' with a component or a feature
                    ProcessSeedComponentArray(llpSeedComps)

                    dispWrapArr = ObjectArrayToDispatchWrapperArray(llpSeedComps)
                    llpDefinition.SeedComponentArray = dispWrapArr

                    ' Update the feature definition
                    swFeature.ModifyDefinition(llpDefinition, swModel, Nothing)
                Case "LOCALCIRPATTERN"
                    ' ILocalCircularPatternFeatureData
                    Debug.Print("    Circular Pattern: " & swFeature.Name)
            End Select
        Next iFeat
    End Sub
End Class

```

```

Dim cpDefinition As LocalCircularPatternFeatureData
cpDefinition = swFeature.GetDefinition
Dim cpSeedComps() As Object
ReDim cpSeedComps(0)
If cpDefinition.AccessSelections(swModel, Nothing) Then
    ' Get the seed components for this pattern
    cpSeedComps = cpDefinition.SeedComponentArray
    ReDim Preserve cpSeedComps(UBound(cpSeedComps))
End If

' Arbitrarily decide whether to replace each seed component
' with a component or a feature
ProcessSeedComponentArray(cpSeedComps)

dispWrapArr = ObjectArrayToDispatchWrapperArray(cpSeedComps)
cpDefinition.SeedComponentArray = dispWrapArr

' Update the feature definition
swFeature.ModifyDefinition(cpDefinition, swModel, Nothing)

Case "DERIVEDLPATTERN"
    ' IDerivedPatternFeatureData
    Debug.Print("    Derived Linear Pattern: " & swFeature.Name)
    Dim dpDefinition As DerivedPatternFeatureData
    dpDefinition = swFeature.GetDefinition
    Dim dpSeedComps(0)
    If dpDefinition.AccessSelections(swModel, Nothing) Then
        ' Get the seed components for this pattern
        dpSeedComps = dpDefinition.SeedComponentArray
        ReDim Preserve dpSeedComps(UBound(dpSeedComps))
    End If

    ' Arbitrarily decide whether to replace each seed component
    ' with a component or a feature
    ProcessSeedComponentArray(dpSeedComps)

    dispWrapArr = ObjectArrayToDispatchWrapperArray(dpSeedComps)
    dpDefinition.SeedComponentArray = dispWrapArr

    ' Update the feature definition
    swFeature.ModifyDefinition(dpDefinition, swModel, Nothing)

Case "DERIVEDCIRPATTERN"
    ' IDerivedPatternFeatureData
    Debug.Print("    Derived Circular Pattern: " & swFeature.Name)
    Dim dcpDefinition As DerivedPatternFeatureData
    dcpDefinition = swFeature.GetDefinition
    Dim dcpSeedComps() As Object
    ReDim dcpSeedComps(0)
    If dcpDefinition.AccessSelections(swModel, Nothing) Then
        ' Get the seed components for this pattern
        dcpSeedComps = dcpDefinition.SeedComponentArray
        ReDim Preserve dcpSeedComps(UBound(dcpSeedComps))
    End If

    ' Arbitrarily decide whether to replace each seed component
    ' with a component or a feature
    ProcessSeedComponentArray(dcpSeedComps)

    dispWrapArr = ObjectArrayToDispatchWrapperArray(dcpSeedComps)
    dcpDefinition.SeedComponentArray = dispWrapArr

    ' Update the feature definition
    swFeature.ModifyDefinition(dcpDefinition, swModel, Nothing)

End Select
Next

End Sub

Private Sub ProcessSeedComponentArray(ByRef vSeedComps As Object())
    Dim replacementSeeds() As Object
    ReDim Preserve replacementSeeds(UBound(vSeedComps))

    ' Iterate over each seed component
    Dim iSeed As Integer
    For iSeed = LBound(vSeedComps) To UBound(vSeedComps)
        Dim swCompFeat As Feature
        swCompFeat = vSeedComps(iSeed)
        Debug.Print("    Seed " & iSeed & ": " & swCompFeat.Name)

        ' Access the seed component represented by the feature
        Dim swComp As Component2
        swComp = swCompFeat.GetSpecificFeature2

        ' Arbitrarily decide whether to replace this seed component
        ' with a component or a feature
        If iSeed Mod 2 = 0 Then
            replacementSeeds(iSeed) = swComp
        Else
            replacementSeeds(iSeed) = swCompFeat
        End If
    Next

    ' Replace the seed array
    vSeedComps = replacementSeeds

```

End Sub

```
Function ObjectArrayToDispatchWrapperArray(ByVal Objects As Object()) As DispatchWrapper()  
    Dim ArraySize As Integer  
    ArraySize = Objects.GetUpperBound(0)  
    Dim d(ArraySize) As DispatchWrapper  
    Dim ArrayIndex As Integer  
    For ArrayIndex = 0 To ArraySize  
        d(ArrayIndex) = New DispatchWrapper(Objects(ArrayIndex))  
    Next  
    Return d  
End Function
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Get and Set Sensor Example (C#)

This example shows how to get a Measurement (dimension) sensor, resets its value to set off an alert, and fire notifications before resetting value.

```
//-----
// Preconditions:
// 1. Open a part document that has a dimension
//    of 2.5 inches and a corresponding Measurement
//    (dimension) sensor that has an alert set to
//    go off if the value of the dimension is
//    reset to > 3 inches.
// 2. Select the Measurement (dimension) sensor
//    of 2.5 inches in the Sensors folder in the
//    FeatureManager design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Enables the Measurement (dimension) sensor alert, if it
//    wasn't previously enabled.
// 2. Sets the value of the dimension to 3.5 inches.
// 3. Triggers the Measurement (dimension) sensor alert; click
//    OK to close it.
// 4. Fires an event when updating the dimension; click
//    OK to close it.
// 5. Examine the FeatureManager design tree, Immediate window,
//    and the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Windows.Forms;
using System.Collections;
namespace SensorsCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public PartDoc swPart;
        public void Main()
        {
            ModelDoc2 swModel;
            SelectionMgr swSelMgr;
            Feature swFeat;
            Sensor swSensor;
            DimensionSensorData swDimSensor;
            DisplayDimension swDisplayDim;
            Dimension swDim;
            double alertValue1;
            double alertValue2;
            double sensorValue;
            int retVal;
            bool retBool;

            Hashtable OpenPart;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Set up event and attach to it
            swPart = (PartDoc)swModel;
            OpenPart = new Hashtable();
            AttachEventHandlers();

            // Get the selected Measurement (dimension) sensor
            // in the Sensors folder in the FeatureManager
            // design tree
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSensor = (Sensor)swFeat.GetSpecificFeature2();
```

```

// Get name of sensor
Debug.Print("Sensor name = " + swFeat.Name);

// Make sure selected sensor is a Measurement
// (dimension) sensor (as of SOLIDWORKS 2009 SP2,
// only Measurement (dimension) sensors are supported);
// if it's not, then exit the macro
if (swSensor == null)
{
    Debug.Print("Selected sensor is not a Measurement (dimension) sensor. Exiting macro.");
    return;
}
// Get type of sensor
switch (swSensor.SensorType)
{
    case (int)swSensorType_e.swSensorSimulation:
        Debug.Print("Sensor type = Simulation");
        break;
    case (int)swSensorType_e.swSensorMassProperty:
        Debug.Print("Sensor type = Mass Property");
        break;
    case (int)swSensorType_e.swSensorDimension:
        Debug.Print("Sensor type = Measurement (dimension)");
        break;
    case (int)swSensorType_e.swSensorInterfaceDetection:
        Debug.Print("Sensor type = Interference Detection");
        break;
}

// Get whether sensor is in an alerted state
Debug.Print("Is an alert currently triggered for this sensor? " + swSensor.SensorAlertState);

//Enable sensor's alert
swSensor.SensorAlertEnabled = true;
Debug.Print ("Is an alert enabled for this sensor? " + swSensor.SensorAlertEnabled);

// Get sensor's alert state
if (swSensor.SensorAlertState)
{
    switch (swSensor.SensorAlertType)
    {
        case (int)swSensorAlertType_e.swSensorAlert_GreaterThan:
            Debug.Print("Sensor alert type = Greater than");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_LessThan:
            Debug.Print("Sensor alert type = Less than");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_Exactly:
            Debug.Print("Sensor alert type = Exactly");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_NotGreaterThan:
            Debug.Print("Sensor alert type = Not greater than");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_NotLessThan:
            Debug.Print("Sensor alert type = Not less than");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_NotExactly:
            Debug.Print("Sensor alert type = Not exactly");
            break;
        case (int)swSensorAlertType_e.swSensorAlert_Between:
            Debug.Print("Sensor alert type = Between");
    }
}

```

```

        break;
    case (int)swSensorAlertType_e.swSensorAlert_NotBetween:
        Debug.Print("Sensor alert type = Not between");
        break;
    case (int)swSensorAlertType_e.swSensorAlert_True:
        Debug.Print("Sensor alert type = True");
        break;
    case (int)swSensorAlertType_e.swSensorAlert_False:
        Debug.Print("Sensor alert type = False");
        break;
    }
    // Get sensor's alert values
    alertValue1 = swSensor.SensorAlertValue1;
    // ISensor::SensorAlertValue2 only valid when sensor
    // alert type is swSensorAlertType_e.swSensorAlert_Between
    alertvalue2 = swSensor.SensorAlertValue2;
    Debug.Print(" Alert value 1 = " + alertValue1);
    Debug.Print(" Alert value 2 = " + alertvalue2);
}
// Set sensor to a different sensor type
swSensor.SensorType = (int)swSensorType_e.swSensorSimulation;
switch (swSensor.SensorType)
{
    case (int)swSensorType_e.swSensorSimulation:
        Debug.Print("Set sensor type to = Simulation");
        break;
    case (int)swSensorType_e.swSensorMassProperty:
        Debug.Print("Set sensor type to = Mass Property");
        break;
    case (int)swSensorType_e.swSensorDimension:
        Debug.Print("Set sensor type to = Measurement (dimension)");
        break;
    case (int)swSensorType_e.swSensorInterfaceDetection:
        Debug.Print("Set sensor type to = Interference Detection");
        break;
}
// Update and evaluate sensor
retBool = swSensor.UpdateSensor();

// Print updated sensor type
switch (swSensor.SensorType)
{
    case (int)swSensorType_e.swSensorSimulation:
        Debug.Print("Sensor updated to type = Simulation");
        break;
    case (int)swSensorType_e.swSensorMassProperty:
        Debug.Print("Sensor updated to type = Mass Property");
        break;
    case (int)swSensorType_e.swSensorDimension:
        Debug.Print("Sensor updated to type = Measurement (dimension)");
        break;
    case (int)swSensorType_e.swSensorInterfaceDetection:
        Debug.Print("Sensor updated to type = Interference Detection");
        break;
}
// Set sensor type back to original type
swSensor.SensorType = (int)swSensorType_e.swSensorDimension;
// Update and evaluate sensor again
retBool = swSensor.UpdateSensor();
// Print updated sensor type
switch (swSensor.SensorType)
{
    case (int)swSensorType_e.swSensorSimulation:

```

```

        Debug.Print("Sensor updated back to type = Simulation");
        break;
    case (int)swSensorType_e.swSensorMassProperty:
        Debug.Print("Sensor updated back to type = Mass Property");
        break;
    case (int)swSensorType_e.swSensorDimension:
        Debug.Print("Sensor updated back to type = Measurement (dimension)");
        break;
    case (int)swSensorType_e.swSensorInterfaceDetection:
        Debug.Print("Sensor updated back to type = Interference Detection");
        break;
}
// Because sensor is a Measurement (dimension) sensor,
// get the sensor's feature data, object, configuration name, and value
if (swSensor is DimensionSensorData)
{
    swDimSensor = (DimensionSensorData)swSensor.GetSensorFeatureData();
    swDisplayDim = (DisplayDimension)swDimSensor.GetDisplayDimension();
    // Get Measurement (dimension) sensor value
    sensorValue = swDimSensor.SensorValue;
    // Convert meters to inches
    Debug.Print("Sensor value: " + (sensorValue * 39.37) + " inches");
    // Get the actual dimension and update
    // to a value that sets off the alert
    swDim = (Dimension)swDisplayDim.GetDimension2(1);
    // Reset and update the Measurement (dimension) to 3.5 inches
    retVal = swDim.SetValue3(3.5, (int)swSetValueInConfiguration_e.swSetValue_UseCurrentSetting, "Configuration");
    retBool = swSensor.UpdateSensor();
    swModel.ForceRebuild3(true);
    // Get Measurement (dimension) sensor value again
    sensorValue = swDimSensor.SensorValue;
    Debug.Print("New sensor value: " + (sensorValue * 39.37) + " inches");
}
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
//Attach to and fire event
public void AttachEventHandlers()
{
    AttachSWEvents();
}
public void AttachSWEvents()
{
    swPart.SensorAlertPreNotify += this.swPart_SensorAlertPreNotify;
}
public int swPart_SensorAlertPreNotify(object SensorIn, int SensorAlertType)
{
    MessageBox.Show("The value of the sensor deviates from its limits.");
    return 1;
}
}
}
}

```

Get and Set Sensor Example (VBA)

This example shows how to get a Measurement (dimension) sensor, resets its value to set off an alert, and fire notifications before resetting value.

```

'-----
' Preconditions:
' 1. Copy Main into the macro.
' 2. Click Insert > Class Module and copy Class1 into that module.
' 3. Open a part document that has a dimension
'    of 2.5 inches and a corresponding Measurement
'    (dimension) sensor that has an alert set to
'    go off if the value of the dimension is
'    reset to > 3 inches.
' 4. Select the Measurement (dimension) sensor
'    of 2.5 inches in the Sensors folder in the
'    FeatureManager design tree.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Enables the Measurement (dimension) sensor alert, if it
'    wasn't previously enabled.
' 2. Sets the value of the dimension to 3.5 inches.
' 3. Triggers the Measurement (dimension) sensor alert; click
'    OK to close it.
' 4. Fires an event when updating the dimension; click
'    OK to close it.
' 5. Examine the FeatureManager design tree, Immediate window,
'    and the graphics area.
'-----

```

Module

Option Explicit

Sub main()

```

    Dim swApp As SldWorks.SldWorks
    Dim swPart As SldWorks.PartDoc
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSensor As SldWorks.Sensor
    Dim swDimSensor As SldWorks.DimensionSensorData
    Dim swDisplayDim As SldWorks.DisplayDimension
    Dim swDim As SldWorks.Dimension
    Dim alertValue1 As Double
    Dim alertValue2 As Double
    Dim sensorValue As Double
    Dim retVal As Long
    Dim swPartEvents As Class1

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Event notification
    Set swPart = swModel
    Set swPartEvents = New Class1
    Set swPartEvents.swPart = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    ' Get the selected Measurement (dimension) sensor
    ' in Sensors folder in FeatureManager design tree
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swSensor = swFeat.GetSpecificFeature2

    ' Get name of sensor
    Debug.Print "Sensor name = " & swFeat.Name

    ' Make sure that the selected sensor is a Measurement
    ' (dimension) sensor (as of SOLIDWORKS 2009 SP2, only
    ' Measurement (dimension) sensors supported);
    ' if it's not, then exit the macro
    If swSensor Is Nothing Then
        Debug.Print "Selected sensor is not a Measurement (dimension) sensor. Exiting macro."
        Exit Sub
    End If

    ' Get type of sensor
    Select Case swSensor.SensorType
        Case swSensor.Simulation
            Debug.Print "Sensor type = Simulation"
        Case swSensor.MassProperty
            Debug.Print "Sensor type = Mass Property"
        Case swSensor.Dimension
            Debug.Print "Sensor type = Measurement (dimension)"
        Case swSensor.InterfaceDetection
            Debug.Print "Sensor type = Interference Detection"
    End Select

    ' Get whether the sensor is in an alerted state
    Debug.Print "Is an alert currently triggered for this sensor ? " & swSensor.SensorAlertState

    ' Enable sensor's alert
    swSensor.SensorAlertEnabled = True
    Debug.Print "Is an alert enabled for this sensor? " & swSensor.SensorAlertEnabled

    ' Get sensor's alert state
    If swSensor.SensorAlertState Then
        Select Case swSensor.SensorAlertType

```

```

        Case swSensorAlert_GreaterThan
            Debug.Print "Sensor alert type = Greater than"
        Case swSensorAlert_LessThan
            Debug.Print "Sensor alert type = Less than"
        Case swSensorAlert_Exactly
            Debug.Print "Sensor alert type = Exactly"
        Case swSensorAlert_NotGreaterThan
            Debug.Print "Sensor alert type = Not greater than"
        Case swSensorAlert_NotLessThan
            Debug.Print "Sensor alert type = Not less than"
        Case swSensorAlert_NotExactly
            Debug.Print "Sensor alert type = Not exactly"
        Case swSensorAlert_Between
            Debug.Print "Sensor alert type = Between"
        Case swSensorAlert_NotBetween
            Debug.Print "Sensor alert type = Not between"
        Case swSensorAlert_True
            Debug.Print "Sensor alert type = True"
        Case swSensorAlert_False
            Debug.Print "Sensor alert type = False"
    End Select

    ' Get sensor's alert values
    alertValue1 = swSensor.SensorAlertValue1

    ' ISensor::SensorAlertValue2 is only valid if sensor
    ' alert type is swSensorAlert_Between
    alertValue2 = swSensor.SensorAlertValue2
    Debug.Print " Alert value 1 = " & alertValue1
    Debug.Print " Alert value 2 = " & alertValue2

End If

' Set sensor to a different sensor type
swSensor.SensorType = swSensorSimulation
Select Case swSensor.SensorType
    Case swSensorSimulation
        Debug.Print "Set sensor type to = Simulation"
    Case swSensorMassProperty
        Debug.Print "Set sensor type to = Mass Property"
    Case swSensorDimension
        Debug.Print "Set sensor type to = Measurement (dimension)"
    Case swSensorInterfaceDetection
        Debug.Print "Set sensor type to = Interference Detection"
End Select

' Update and evaluate sensor
swSensor.UpdateSensor

' Set sensor type back to original type
swSensor.SensorType = swSensorDimension

' Update and evaluate sensor again
swSensor.UpdateSensor

' Print updated sensor type
Select Case swSensor.SensorType
    Case swSensorSimulation
        Debug.Print "Sensor updated back to type = Simulation"
    Case swSensorMassProperty
        Debug.Print "Sensor updated back to type = Mass Property"
    Case swSensorDimension
        Debug.Print "Sensor updated back to type = Measurement (dimension)"
    Case swSensorInterfaceDetection
        Debug.Print "Sensor updated back to type = Interference Detection"
End Select

' Because sensor is a Measurement (dimension) sensor,
' get the sensor's feature data, object, configuration name, and value
If TypeOf swSensor Is SldWorks.DimensionSensorData Then
    Set swDimSensor = swSensor.GetSensorFeatureData

    ' Get Measurement (dimension) sensor value
    sensorValue = swDimSensor.sensorValue

    ' Convert meters to inches
    Debug.Print "Sensor value: " & (sensorValue * 39.37) & " inches"

    ' Get the actual dimension and update it
    ' to a value that sets off the alert
    Set swDisplayDim = swDimSensor.GetDisplayDimension
    Set swDim = swDisplayDim.GetDimension2(1)
    retVal = swDim.SetValue3(3.5, swSetValue_UseCurrentSetting, Nothing)
    swSensor.UpdateSensor
    swModel.ForceRebuild3 (True)

    ' Get Measurement (dimension) sensor value again
    sensorValue = swDimSensor.sensorValue

    ' Convert meters to inches
    Debug.Print "New sensor value: " & (sensorValue * 39.37) & " inches"

End If

End Sub

```

[Back to top](#)

Class

Option Explicit

```
Public WithEvents swPart As SldWorks.PartDoc
Private Function swPart_SensorAlertPreNotify(ByVal SensorIn As Object, ByVal SensorAlertType As Long) As Long
    MsgBox "The value of the sensor deviates from its limits."
End Function
```

[Back to top](#)

Get and Set Sensor Example (VB.NET)

This example shows how to get a Measurement (dimension) sensor, resets its value to set off an alert, and fire notification before resetting the value.

```

'-----
' Preconditions:
' 1. Open a part document that has a dimension
'   of 2.5 inches and a corresponding Measurement
'   (dimension) sensor that has an alert set to
'   go off if the value of the dimension is
'   reset to > 3 inches.
' 2. Select the Measurement (dimension) sensor
'   of 2.5 inches in the Sensors folder in the
'   FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Enables the Measurement (dimension) sensor alert, if it
'   wasn't previously enabled.
' 2. Sets the value of the dimension to 3.5 inches.
' 3. Triggers the Measurement (dimension) sensor alert; click
'   OK to close it.
' 4. Fires an event when updating the dimension; click
'   OK to close it.
' 5. Examine the FeatureManager design tree, Immediate window,
'   and the graphics area.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Imports System.Collections
```

```
Partial Class SolidWorksMacro
```

```
    Public WithEvents swPart As PartDoc
```

```
    Public Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swSensor As Sensor
        Dim swDimSensor As DimensionSensorData
        Dim swDisplayDim As DisplayDimension
        Dim swDim As Dimension
        Dim alertValue1 As Double
        Dim alertvalue2 As Double
        Dim sensorValue As Double
        Dim retVal As Integer
        Dim bool As Boolean
        Dim openPart As Hashtable

```

```

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

```

```

        ' Set up event handler
        swPart = swModel
        openPart = New Hashtable
        AttachEventHandlers()

```

```

        ' Get the selected Measurement (dimension) sensor
        ' in Sensors folder in FeatureManager design tree
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swSensor = swFeat.GetSpecificFeature2
        ' Get name of sensor
        Debug.Print("Sensor name = " & swFeat.Name)

```

```

' Make sure selected sensor is a Measurement
' (dimension) sensor (as of SOLIDWORKS 2009 SP2, only Measurement
' (dimension) sensors are supported); if it's not, then exit
' the macro
If swSensor Is Nothing Then
    Debug.Print("Selected sensor is not a Measurement (dimension) sensor. Exiting macro.")
    Exit Sub
End If

' Get type of sensor
Select Case swSensor.SensorType
    Case swSensorType_e.swSensorSimulation
        Debug.Print("Sensor type = Simulation")
    Case swSensorType_e.swSensorMassProperty
        Debug.Print("Sensor type = Mass Property")
    Case swSensorType_e.swSensorDimension
        Debug.Print("Sensor type = Measurement (dimension)")
    Case swSensorType_e.swSensorInterfaceDetection
        Debug.Print("Sensor type = Interference Detection")
End Select

' Get whether the sensor is in an alerted state
Debug.Print ("Is an alert currently triggered for this sensor? " & swSensor.SensorAlertState)

' Enable sensor's alert
Debug.Print ("Is an alert enabled for this sensor? " & swSensor.SensorAlertEnabled)

' Get sensor's alert state
If swSensor.SensorAlertState Then
    Select Case swSensor.SensorAlertType
        Case swSensorAlertType_e.swSensorAlert_GreaterThan
            Debug.Print("Sensor alert type = Greater than")
        Case swSensorAlertType_e.swSensorAlert_LessThan
            Debug.Print("Sensor alert type = Less than")
        Case swSensorAlertType_e.swSensorAlert_Exactly
            Debug.Print("Sensor alert type = Exactly")
        Case swSensorAlertType_e.swSensorAlert_NotGreaterThan
            Debug.Print("Sensor alert type = Not greater than")
        Case swSensorAlertType_e.swSensorAlert_NotLessThan
            Debug.Print("Sensor alert type = Not less than")
        Case swSensorAlertType_e.swSensorAlert_NotExactly
            Debug.Print("Sensor alert type = Not exactly")
        Case swSensorAlertType_e.swSensorAlert_Between
            Debug.Print("Sensor alert type = Between")
        Case swSensorAlertType_e.swSensorAlert_NotBetween
            Debug.Print("Sensor alert type = Not between")
        Case swSensorAlertType_e.swSensorAlert_True
            Debug.Print("Sensor alert type = True")
        Case swSensorAlertType_e.swSensorAlert_False
            Debug.Print("Sensor alert type = False")
    End Select

    ' Get sensor's alert values
    alertValue1 = swSensor.SensorAlertValue1
    ' ISensor::SensorAlertValue2 only valid when sensor
    ' alert type is swSensorAlertType_e.swSensorAlert_Between
    alertValue2 = swSensor.SensorAlertValue2
    Debug.Print(" Alert value 1 = " & alertValue1)
    Debug.Print(" Alert value 2 = " & alertValue2)
End If

' Set sensor to a different sensor type

```

```

swSensor.SensorType = swSensorType_e.swSensorSimulation
Select Case swSensor.SensorType
    Case swSensorType_e.swSensorSimulation
        Debug.Print("Set sensor type to = Simulation")
    Case swSensorType_e.swSensorMassProperty
        Debug.Print("Set sensor type to = Mass Property")
    Case swSensorType_e.swSensorDimension
        Debug.Print("Set sensor type to = Measurement (dimension)")
    Case swSensorType_e.swSensorInterfaceDetection
        Debug.Print("Set sensor type to = Interference Detection")
End Select

' Update and evaluate sensor
bool = swSensor.UpdateSensor()

' Print updated sensor type
Select Case swSensor.SensorType
    Case swSensorType_e.swSensorSimulation
        Debug.Print("Sensor updated to type = Simulation")
    Case swSensorType_e.swSensorMassProperty
        Debug.Print("Sensor updated to type = Mass Property")
    Case swSensorType_e.swSensorDimension
        Debug.Print("Sensor updated to type = Measurement (dimension)")
    Case swSensorType_e.swSensorInterfaceDetection
        Debug.Print("Sensor updated to type = Interference Detection")
End Select

' Set sensor type back to original type
swSensor.SensorType = swSensorType_e.swSensorDimension

' Update and evaluate sensor again
bool = swSensor.UpdateSensor()
' Print updated sensor type
Select Case swSensor.SensorType
    Case swSensorType_e.swSensorSimulation
        Debug.Print("Sensor updated back to type = Simulation")
    Case swSensorType_e.swSensorMassProperty
        Debug.Print("Sensor updated back to type = Mass Property")
    Case swSensorType_e.swSensorDimension
        Debug.Print("Sensor updated back to type = Measurement (dimension)")
    Case swSensorType_e.swSensorInterfaceDetection
        Debug.Print("Sensor updated back to type = Interference Detection")
End Select

' Because sensor is a Measurement (dimension) sensor,
' get the sensor's feature data, object, configuration name, and value
If TypeOf swSensor Is DimensionSensorData Then
    swDimSensor = swSensor.GetSensorFeatureData
    ' Get Measurement (dimension) sensor value
    sensorValue = swDimSensor.SensorValue
    'Convert meters to inches
    Debug.Print("Sensor value: " + Str(sensorValue * 39.37) + " inches")
    ' Get the actual dimension and update
    ' to a value that sets off the alert
    swDisplayDim = swDimSensor.GetDisplayDimension()
    swDim = swDisplayDim.GetDimension2(1)
    retVal = swDim.SetValue3(3.5, swSetValueInConfiguration_e.swSetValue_UseCurrentSetting, "Configuration")
    bool = swSensor.UpdateSensor()
    swModel.ForceRebuild3(True)
    ' Get Measurement (dimension) sensor value again
    sensorValue = swDimSensor.SensorValue
    Debug.Print("New sensor value: " + Str(sensorValue * 39.37) + " inches")

```

```
End If
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
Sub AttachEventHandlers()
    AttachSWEvents()
End Sub
```

```
Sub AttachSWEvents()
    AddHandler swPart.SensorAlertPreNotify, AddressOf Me.swPart_SensorAlertPreNotify
End Sub
```

```
Function swPart_SensorAlertPreNotify(ByVal SensorIn As Object, ByVal SensorAlertType As Integer) As Integer
    MsgBox("The value of the sensor deviates from its limits.")
End Function
```

```
End Class
```

Get and Set Sheet Properties Example (VBA)

This example shows how to get the sheet properties and how to switch projection.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block1.sldprt and make
'    a drawing with front, side, and top views.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Switches projection to third angle from first angle or vice versa.
' 2. Examine the graphics area and Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swDraw As SldWorks.DrawingDoc
    Dim swSheet As SldWorks.Sheet
    Dim vSheetProps As Variant
    Dim bFirstAng As Boolean
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swDraw = swApp.ActiveDoc
    Set swSheet = swDraw.GetCurrentSheet

    ' Get current sheet properties
    vSheetProps = swSheet.GetProperties

    ' Current sheet properties
    Debug.Print "Name = " + swSheet.GetName
    Debug.Print "  TemplateName          = " & swSheet.GetTemplateName
    Debug.Print "  PaperSize              = " & vSheetProps(0)
    Debug.Print "  TemplateIn               = " & vSheetProps(1)
    Debug.Print "  Scale1                   = " & vSheetProps(2)
    Debug.Print "  scale2                   = " & vSheetProps(3)
    Debug.Print "  FirstAngle               = " & vSheetProps(4)
    Debug.Print "  Width                    = " & vSheetProps(5)
    Debug.Print "  Height                   = " & vSheetProps(6)

    ' Switch projection property
    bFirstAng = vSheetProps(4)
    vSheetProps(4) = Not bFirstAng

    ' Apply updated sheet properties
    swSheet.SetProperties vSheetProps(0), vSheetProps(1), vSheetProps(2), vSheetProps(3), vSheetProps(4), vSheetProps(5), vSheetProps(6)

    ' Current projection property
    Debug.Print " "
    Debug.Print "  New FirstAngle setting    = " & vSheetProps(4)

    swDraw.EditRebuild
End Sub
```

Get and Set Spline Handles Example (C#)

This example shows how to get and set the properties of spline handles in a 2D spline.

```
//-----
// Preconditions:
// 1. Verify that the specified document template exists.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates Sketch1 in the graphics area and FeatureManager design tree.
// 2. Modifies some of the spline handles of Sketch1.
// 3. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetSetSplineHandle_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SelectionMgr swSelMgr;
        ModelDocExtension swModelDocExt;
        SketchSpline swSpline;
        SketchSegment skSegment;
        object[] objSplineHandle;
        SplineHandle swSplineHandle;
        bool boolstatus;

        int i;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2015\\templates\\Part.prtdot", 0, 0, 0);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;

            object pointArray = null;
            double[] points = new double[18];
            points[0] = -0.0725658847190971;
            points[1] = -0.0284590725570979;
            points[2] = 0;
            points[3] = -0.0437940929359115;
            points[4] = 0.0215374317625674;
            points[5] = 0;
            points[6] = -0.0245140262770747;
            points[7] = -0.026920232075895;
            points[8] = 0;
            points[9] = 0.0273938454967038;
            points[10] = -0.0414386885537397;
            points[11] = 0;
            points[12] = 0.0413348167730874;
            points[13] = 0.0386761654855832;
            points[14] = 0;
            points[15] = 0.101251331620574;
            points[16] = 0.0438481259864147;
            points[17] = 0;
            pointArray = points;

            skSegment = swModel.SketchManager.CreateSpline((pointArray));
            swModel.SketchManager.InsertSketch(true);

            boolstatus = swModelDocExt.SelectByID2("Spline1@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, false, 0, null, 0);
            swSpline = (SketchSpline)swSelMgr.GetSelectedObject6(1, 0);

            // Get the handles on the spline
            objSplineHandle = (object[])swSpline.GetSplineHandles();

            for (i = 0; i <= objSplineHandle.GetUpperBound(0); i++)
            {
                swSplineHandle = (SplineHandle)objSplineHandle[i];
                Debug.Print("Spline handle " + swSplineHandle.SplinePointNumber);
                Debug.Print(" X: " + swSplineHandle.X * 1000);
                Debug.Print(" Y: " + swSplineHandle.Y * 1000);
                Debug.Print(" Z: " + swSplineHandle.Z * 1000);
            }
        }
    }
}
```

```

    if ((i == 0))
    {
        swSplineHandle.X = -62.33246528 / 1000;
        swSplineHandle.Y = -14.71944444 / 1000;
    }

    Debug.Print(" Tangent magnitude: " + swSplineHandle.get_TangentMagnitude((int)swTangentMagnitudeDirection_e.swTangentMagnitudeDirection);
    Debug.Print(" Tangent radial direction: " + swSplineHandle.TangentRadialDirection);

    if ((i == 2))
    {
        swSplineHandle.set_TangentMagnitude((int)swTangentMagnitudeDirection_e.swTangentMagnitudeDirection1, swSplineHandle.TangentRadialDirection + 0.5);
    }

    Debug.Print(" Curvature: " + swSplineHandle.Curvature);
    Debug.Print(" Radius of curvature: " + swSplineHandle.RadiusOfCurvature);

    if ((i == 3))
    {
        swSplineHandle.RadiusOfCurvature = swSplineHandle.RadiusOfCurvature / 2;
    }

    if ((i == 3))
    {
        Debug.Print(" Tangent driving? " + swSplineHandle.TangentDriving);

        if ((swSplineHandle.TangentDriving))
        {
            swSplineHandle.TangentDriving = false;
        }
        else
        {
            swSplineHandle.TangentDriving = true;
        }

        swSplineHandle.Reset();
    }
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Get and Set Spline Handles Example (VBA)

This example shows how to get and set the properties of spline handles in a 2D spline.

Preconditions:

1. Verify that the specified document template exists.
2. Open an Immediate window.

Postconditions:

1. Creates **Sketch1** in the graphics area and FeatureManager design tree.
2. Modifies some of the spline handles of **Sketch1**.
3. Inspect the Immediate window.

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSpline As SldWorks.SketchSpline
Dim skSegment As SldWorks.SketchSegment
Dim swSplineHandle As Variant
Dim boolstatus As Boolean
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 17) As Double
    points(0) = -7.25658847190971E-02
    points(1) = -2.84590725570979E-02
    points(2) = 0
    points(3) = -4.37940929359115E-02
    points(4) = 2.15374317625674E-02
    points(5) = 0
    points(6) = -2.45140262770747E-02
    points(7) = -0.026920232075895
    points(8) = 0
    points(9) = 2.73938454967038E-02
    points(10) = -4.14386885537397E-02
    points(11) = 0
    points(12) = 4.13348167730874E-02
    points(13) = 3.86761654855832E-02
    points(14) = 0
    points(15) = 0.101251331620574
    points(16) = 4.38481259864147E-02
    points(17) = 0
    pointArray = points

    Set skSegment = swModel.SketchManager.CreateSpline((pointArray))
    swModel.SketchManager.InsertSketch True

    boolstatus = swModelDocExt.SelectByID("Spline1@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swSpline = swSelMgr.GetSelectedObject6(1, 0)

    ' Get the handles on the spline
    swSplineHandle = swSpline.GetSplineHandles()

    For i = 0 To UBound(swSplineHandle)

        Debug.Print "Spline handle " & swSplineHandle(i).SplinePointNumber
        Debug.Print " X: " & swSplineHandle(i).X * 1000
        Debug.Print " Y: " & swSplineHandle(i).Y * 1000
        Debug.Print " Z: " & swSplineHandle(i).Z * 1000

        If (i = 0) Then

            swSplineHandle(i).X = -62.33246528 / 1000
            swSplineHandle(i).Y = -14.71944444 / 1000

        End If

        Debug.Print " Tangent magnitude: " & swSplineHandle(i).TangentMagnitude(swTangentMagnitudeDirection1)
        Debug.Print " Tangent radial direction: " & swSplineHandle(i).TangentRadialDirection

        If (i = 2) Then

            swSplineHandle(i).TangentMagnitude(swTangentMagnitudeDirection1) =
            (swSplineHandle(i).TangentMagnitude(swTangentMagnitudeDirection1)) + 0.02
            swSplineHandle(i).TangentRadialDirection = swSplineHandle(i).TangentRadialDirection + 0.5

        End If

        Debug.Print " Curvature: " & swSplineHandle(i).Curvature
        Debug.Print " Radius of curvature: " & swSplineHandle(i).RadiusOfCurvature

        If (i = 3) Then

            swSplineHandle(i).RadiusOfCurvature = swSplineHandle(i).RadiusOfCurvature / 2

```

```
End If
If (i = 3) Then
    Debug.Print " Tangent driving? " & swSplineHandle(i).TangentDriving
    If (swSplineHandle(i).TangentDriving) Then
        swSplineHandle(i).TangentDriving = False
    Else
        swSplineHandle(i).TangentDriving = True
    End If
    swSplineHandle(i).Reset
End If
Next
End Sub
```

Get and Set Spline Handles Example (VB.NET)

This example shows how to get and set the properties of spline handles in a 2D spline.

```

'-----
' Preconditions:
' 1. Verify that the specified document template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates Sketch1 in the graphics area and FeatureManager design tree.
' 2. Modifies some of the spline handles of Sketch1.
' 3. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swModelDocExt As ModelDocExtension
    Dim swSpline As SketchSpline
    Dim skSegment As SketchSegment
    Dim swSplineHandle As Object
    Dim boolstatus As Boolean
    Dim i As Integer

```

```
Sub main()
```

```

    swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)
    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager
    swModelDocExt = swModel.Extension

```

```

    Dim pointArray As Object
    Dim points() As Double = New Double(17) {}
    points(0) = -0.0725658847190971
    points(1) = -0.0284590725570979
    points(2) = 0
    points(3) = -0.0437940929359115
    points(4) = 0.0215374317625674
    points(5) = 0
    points(6) = -0.0245140262770747
    points(7) = -0.026920232075895
    points(8) = 0
    points(9) = 0.0273938454967038
    points(10) = -0.0414386885537397
    points(11) = 0
    points(12) = 0.0413348167730874
    points(13) = 0.0386761654855832
    points(14) = 0
    points(15) = 0.101251331620574
    points(16) = 0.0438481259864147
    points(17) = 0
    pointArray = points

```

```

    skSegment = swModel.SketchManager.CreateSpline((pointArray))
    swModel.SketchManager.InsertSketch(True)

```

```

    boolstatus = swModelDocExt.SelectByID2("Spline1@Sketch1", "EXTSKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    swSpline = swSelMgr.GetSelectedObject6(1, 0)

```

```

' Get the handles on the spline
swSplineHandle = swSpline.GetSplineHandles()

```

```
For i = 0 To UBound(swSplineHandle)
```

```

    Debug.Print("Spline handle " & swSplineHandle(i).SplinePointNumber)
    Debug.Print(" X: " & swSplineHandle(i).X * 1000)
    Debug.Print(" Y: " & swSplineHandle(i).Y * 1000)
    Debug.Print(" Z: " & swSplineHandle(i).Z * 1000)

```

```
    If (i = 0) Then
```

```

        swSplineHandle(i).X = -62.33246528 / 1000
        swSplineHandle(i).Y = -14.71944444 / 1000

```

```
    End If
```

```

    Debug.Print(" Tangent magnitude: " & swSplineHandle(i).TangentMagnitude(swTangentMagnitudeDirection_e.swTangentMagnitudeDirection_e))
    Debug.Print(" Tangent radial direction: " & swSplineHandle(i).TangentRadialDirection)

```

```
    If (i = 2) Then
```

```
swSplineHandle(i).TangentMagnitude(swTangentMagnitudeDirection_e.swTangentMagnitudeDirection1) = (swSplineHandle(i).swSplineHandle(i).TangentRadialDirection = swSplineHandle(i).TangentRadialDirection + 0.5
```

```
End If
```

```
Debug.Print(" Curvature: " & swSplineHandle(i).Curvature)  
Debug.Print(" Radius of curvature: " & swSplineHandle(i).RadiusOfCurvature)
```

```
If (i = 3) Then
```

```
swSplineHandle(i).RadiusOfCurvature = swSplineHandle(i).RadiusOfCurvature / 2
```

```
End If
```

```
If (i = 3) Then
```

```
Debug.Print(" Tangent driving? " & swSplineHandle(i).TangentDriving)
```

```
If (swSplineHandle(i).TangentDriving) Then  
swSplineHandle(i).TangentDriving = False
```

```
Else  
swSplineHandle(i).TangentDriving = True
```

```
End If
```

```
swSplineHandle(i).Reset()
```

```
End If
```

```
Next
```

```
End Sub
```

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

```
End Class
```

Get and Set Spline Properties Example (VBA)

This example shows how to get and set various spline properties.

```
'-----
' Preconditions: Open a new part, create a
' 2D spline, and select the spline in the
' graphics area.
'
' Postconditions:
' 1. Gets and sets properties of the selected spline.
' 2. Examine the graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSpline As SldWorks.SketchSpline
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Spline1", "SKETCHSEGMENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swSpline = swSelMgr.GetSelectedObject6(1, 0)

    If (swSpline.ShowSplineHandles) Then
        swSpline.ShowSplineHandles = False
    Else
        swSpline.ShowSplineHandles = True
    End If

    swModel.WindowRedraw 'Required after using swSpine.ShowSplineHandles

    If (swSpline.DisplayControlPolygon) Then
        swSpline.DisplayControlPolygon = False
    Else
        swSpline.DisplayControlPolygon = True
    End If

    If (swSpline.ShowInflectionPoints) Then
        swSpline.ShowInflectionPoints = False
    Else
        swSpline.ShowInflectionPoints = True
    End If

    If (swSpline.ShowMinimumRadius) Then
        swSpline.ShowMinimumRadius = False
    Else
        swSpline.ShowMinimumRadius = True
    End If

    If (swSpline.ShowCurvatureCombs) Then
        swSpline.ShowCurvatureCombs = False
    Else
        swSpline.ShowCurvatureCombs = True
    End If

    If (swSpline.Proportional) Then
        swSpline.Proportional = False
    Else
        swSpline.Proportional = True
    End If

    swModel.WindowRedraw

End Sub
```

Get and Set the State of the Flyout FeatureManager Design Tree Example (C#)

This example shows how to to display, expand, and hide the flyout FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Click OK to close each of the three message boxes after examining the
// graphics area.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Windows.Forms;

namespace FlyoutFeatureTreeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        int status;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = (int)swModelDocExt.FlyoutFeatureTreeVisibility;

            Debug.Print("-----Flyout state at start-----");
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_Hidden)
                Debug.Print("FlyoutFeatureTree Hidden");
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded)
                Debug.Print("FlyoutFeatureTree Expanded");
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded)
                Debug.Print("FlyoutFeatureTree UnExpanded");

            swModelDocExt.FlyoutFeatureTreeVisibility = (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded;
            status = swModelDocExt.FlyoutFeatureTreeVisibility;
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded)
                MessageBox.Show("Flyout state is now swFlyoutFeatureTree_ShownExpanded.");

            swModelDocExt.FlyoutFeatureTreeVisibility = (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded;
            status = swModelDocExt.FlyoutFeatureTreeVisibility;
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded)
                MessageBox.Show("Flyout state is now swFlyoutFeatureTree_ShownUnExpanded.");

            swModelDocExt.FlyoutFeatureTreeVisibility = (int)swFeatureTreeState_e.swFlyoutFeatureTree_Hidden;
            status = swModelDocExt.FlyoutFeatureTreeVisibility;
            if (status == (int)swFeatureTreeState_e.swFlyoutFeatureTree_Hidden)
                MessageBox.Show("Flyout state is now swFlyoutFeatureTree_Hidden.");
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get and Set the State of the Flyout FeatureManager Design Tree Example (VBA)

This example shows how to display, expand, and hide the flyout FeatureManager design tree.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Click OK to close each of the three message boxes after examining the
'    graphics area.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim ext As SldWorks.ModelDocExtension
Dim Status As Long

Sub main()
    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    Set ext = Part.Extension
    Status = ext.FlyoutFeatureTreeVisibility

    Debug.Print "-----Flyout state at start-----"
    If Status = swFlyoutFeatureTree_Hidden Then Debug.Print "FlyoutFeatureTree Hidden"
    If Status = swFlyoutFeatureTree_ShownExpanded Then Debug.Print "FlyoutFeatureTree Expanded"
    If Status = swFlyoutFeatureTree_ShownUnExpanded Then Debug.Print "FlyoutFeatureTree UnExpanded"

    ext.FlyoutFeatureTreeVisibility = swFlyoutFeatureTree_ShownExpanded
    Status = ext.FlyoutFeatureTreeVisibility
    If Status = swFlyoutFeatureTree_ShownExpanded Then MsgBox ("Flyout state is now swFlyoutFeatureTree_ShownExpanded.")

    ext.FlyoutFeatureTreeVisibility = swFlyoutFeatureTree_ShownUnExpanded
    Status = ext.FlyoutFeatureTreeVisibility
    If Status = swFlyoutFeatureTree_ShownUnExpanded Then MsgBox ("Flyout state is now swFlyoutFeatureTree_ShownUnExpanded.")

    ext.FlyoutFeatureTreeVisibility = swFlyoutFeatureTree_Hidden
    Status = ext.FlyoutFeatureTreeVisibility
    If Status = swFlyoutFeatureTree_Hidden Then MsgBox ("Flyout state is now swFlyoutFeatureTree_Hidden.")
End Sub

```

Get and Set the State of the Flyout FeatureManager Design Tree Example (VB.NET)

This example shows how to display, expand, and hide the flyout FeatureManager design tree.

```
'-----
' Preconditions:
' 1. Open a SOLIDWORKS document.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Click OK to close each of the three message boxes after examining the
'    graphics area.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim status As Integer

    Public Sub Main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        status = swModelDocExt.FlyoutFeatureTreeVisibility

        Debug.Print("-----Flyout state at start-----")
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_Hidden Then Debug.Print("FlyoutFeatureTree Hidden")
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded Then Debug.Print("FlyoutFeatureTree Expanded")
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded Then Debug.Print("FlyoutFeatureTree UnExpanded")

        swModelDocExt.FlyoutFeatureTreeVisibility = swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded
        status = swModelDocExt.FlyoutFeatureTreeVisibility
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_ShownExpanded Then MsgBox("Flyout state is now swFlyoutFeatureTree_ShownExpanded.")

        swModelDocExt.FlyoutFeatureTreeVisibility = swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded
        status = swModelDocExt.FlyoutFeatureTreeVisibility
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_ShownUnExpanded Then MsgBox("Flyout state is now swFlyoutFeatureTree_ShownUnExpanded.")

        swModelDocExt.FlyoutFeatureTreeVisibility = swFeatureTreeState_e.swFlyoutFeatureTree_Hidden
        status = swModelDocExt.FlyoutFeatureTreeVisibility
        If status = swFeatureTreeState_e.swFlyoutFeatureTree_Hidden Then MsgBox("Flyout state is now swFlyoutFeatureTree_Hidden.")

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class
```

Get and Set Sunlight Source Property Values Example (C#)

This example shows how to get and set sunlight properties.

```
//-----
// Preconditions:
// 1. Open a part with a sunlight source.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window for the sunlight properties.
// -----

using
Microsoft.VisualBasic;
using

System;
using

System.Collections;
using

System.Collections.Generic;
using

System.Data;
using

System.Diagnostics;
using

SolidWorks.Interop.sldworks;
using

SolidWorks.Interop.swconst;
using

System.Runtime.InteropServices;
namespace
SunlightProperties_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModelDoc;
        ModelDocExtension swModelDocExt;
        bool retVal;
        MathVector NorthDirection;
        string DateTime;
        object vVector;
        double[] nVector = new double[3];
        MathUtility swMathUtil;
        double NorthLatitude;
        double EastLongitude;
        double TimeZone;
        double Haze;
        double SunDiameter;
        int GroundAlbedo;
        double SkyGamma;
        public void Main()
        {
            swModelDoc = (
                ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModelDoc.Extension;

            //Get sunlight properties
            retVal = swModelDocExt.GetSunLightSourcePropertyValues(
                out NorthDirection, out NorthLatitude, out EastLongitude, out TimeZone, out DateTime);
            retVal = swModelDocExt.GetSunLightAdvancedPropertyValues(
                out Haze, out SunDiameter, out GroundAlbedo, out SkyGamma);

            Debug.Print("North direction: " + ((double[])(NorthDirection.ArrayData))[0] + "," + ((double[])(NorthDirection.ArrayData))[1] + "," + ((double[])(NorthDirection.ArrayData))[2]);
            Debug.Print("North latitude: " + NorthLatitude);
            Debug.Print("East longitude: " + EastLongitude);
            Debug.Print("Time zone: " + TimeZone);
            Debug.Print("Date and time: " + DateTime);
            Debug.Print("Haze (0.0 - 1.0): " + Haze);
        }
    }
}
```

```

Debug.Print("Sun diameter (0.01 - 21474836.47): " + SunDiameter);
Debug.Print("RGB for ground albedo: " + GroundAlbedo);
Debug.Print("Sky Gamma (0.1 = 100.0): " + SkyGamma);
Debug.Print("Minutes of sunlight: " + swModelDocExt.get_SunLightInformation((int)swSunLightInfoType_e.swSunLight_LengthOfDay));
Debug.Print("Sunrise (hours from midnight): " +
swModelDocExt.get_SunLightInformation((int)swSunLightInfoType_e.swSunLight_Sunrise));
Debug.Print("Sunset (hours from midnight): " + swModelDocExt.get_SunLightInformation((int)swSunLightInfoType_e.swSunLight_Sunset));
//Set sunlight source property values
swMathUtil = (
MathUtility)swApp.GetMathUtility();
nVector[0] = 1;
nVector[1] = 0;
nVector[2] = 0;
vVector = nVector;
NorthDirection = (
MathVector)swMathUtil.CreateVector((vVector));
DateTime =
"11/9/2012 2:48:13 PM";
NorthLatitude = NorthLatitude + 0.1;
EastLongitude = EastLongitude + 0.1;
TimeZone = TimeZone + 0.5;
retval = swModelDocExt.SetSunLightSourcePropertyValues(NorthDirection, NorthLatitude, EastLongitude, TimeZone, DateTime);
}
public SldWorks swApp;
}
}

```

Get and Set Sunlight Source Property Values Example (VBA)

This example shows how to get and set sunlight properties.

```

'-----
' Preconditions:
' 1. Open a part with a sunlight source.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window for the sunlight properties.
'-----

Dim swApp           As SldWorks.SldWorks
Dim swModelDoc      As SldWorks.ModelDoc2
Dim swModelDocExt   As SldWorks.ModelDocExtension
Dim retVal          As Boolean
Dim NorthDirection As SldWorks.MathVector
Dim DateTime        As String
Dim vVector         As Variant
Dim nVector(2)      As Double
Dim swMathUtil      As SldWorks.MathUtility
Dim NorthLatitude   As Double
Dim EastLongitude   As Double
Dim TimeZone        As Double
Dim Haze As Double
Dim SunDiameter As Double
Dim GroundAlbedo As Long
Dim SkyGamma As Double

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swModelDocExt = swModelDoc.Extension

    'Get sunlight property values
    retVal = swModelDocExt.GetSunLightSourcePropertyValues(NorthDirection, NorthLatitude, EastLongitude, TimeZone, DateTime)
    retVal = swModelDocExt.GetSunLightAdvancedPropertyValues(Haze, SunDiameter, GroundAlbedo, SkyGamma)

    Debug.Print "North direction: " & NorthDirection.ArrayData(0) & ", " & NorthDirection.ArrayData(1) & ", " &
NorthDirection.ArrayData(2)
    Debug.Print "North latitude: " & NorthLatitude
    Debug.Print "East longitude: " & EastLongitude
    Debug.Print "Time Zone: " & TimeZone
    Debug.Print "Date and time: " & DateTime
    Debug.Print "Haze (0.0 - 1.0): " & Haze
    Debug.Print "Sun diameter (0.01 - 21474836.47): " & SunDiameter
    Debug.Print "RGB for ground albedo: " & GroundAlbedo
    Debug.Print "Sky Gamma (0.1 = 100.0): " & SkyGamma
    Debug.Print "Minutes of sunlight: " & swModelDocExt.SunLightInformation(swSunlight_LengthOfDay)
    Debug.Print "Sunrise (hours from midnight): " & swModelDocExt.SunLightInformation(swSunlight_Sunrise)
    Debug.Print "Sunset (hours from midnight): " & swModelDocExt.SunLightInformation(swSunlight_Sunset)

    'Set sunlight source property values
    Set swMathUtil = swApp.GetMathUtility
    nVector(0) = 1: nVector(1) = 0: nVector(2) = 0
    vVector = nVector
    Set NorthDirection = swMathUtil.CreateVector((vVector))

    DateTime = "11/9/2012 2:48:13 PM"
    NorthLatitude = NorthLatitude + 0.1
    EastLongitude = EastLongitude + 0.1
    TimeZone = TimeZone + 0.5
    retVal = swModelDocExt.SetSunLightSourcePropertyValues(NorthDirection, NorthLatitude, EastLongitude, TimeZone, DateTime)

End Sub

```

Get and Set Sunlight Source Property Values Example (VB.NET)

This example shows how to get and set sunlight properties.

```
' -----
' Preconditions:
' 1. Open a part with a sunlight source.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window for the sunlight properties.
' -----

Imports
SolidWorks.Interop.sldworks
Imports
SolidWorks.Interop.swconst
Imports
System.Runtime.InteropServices
Imports
System
Imports
System.Diagnostics

Partial
Class SolidWorksMacro
Dim swModelDoc As ModelDoc2
Dim swModelDocExt As ModelDocExtension
Dim retval As Boolean
Dim NorthDirection As MathVector
Dim DateTime As String
Dim vVector As Object
Dim nVector(2) As Double
Dim swMathUtil As MathUtility
Dim NorthLatitude As Double
Dim EastLongitude As Double
Dim TimeZone As Double
Dim Haze As Double
Dim SunDiameter As Double
Dim GroundAlbedo As Integer
Dim SkyGamma As Double
Sub main()
    swModelDoc = swApp.ActiveDoc
    swModelDocExt = swModelDoc.Extension

    'Get sunlight property values
    retval = swModelDocExt.GetSunLightSourcePropertyValues(NorthDirection, NorthLatitude, EastLongitude, TimeZone, DateTime)
    retval = swModelDocExt.GetSunLightAdvancedPropertyValues(Haze, SunDiameter, GroundAlbedo, SkyGamma)
    Debug.Print(
        "North direction: " & NorthDirection.ArrayData(0) & ", " & NorthDirection.ArrayData(1) & ", " & NorthDirection.ArrayData(2))
    Debug.Print(
        "North latitude: " & NorthLatitude)
    Debug.Print(
        "East longitude: " & EastLongitude)
    Debug.Print(
        "Time zone: " & TimeZone)
    Debug.Print(
        "Date and time: " & DateTime)
    Debug.Print(
        "Haze (0.0 - 1.0): " & Haze)
    Debug.Print(
        "Sun diameter (0.01 - 21474836.47): " & SunDiameter)
    Debug.Print(
        "RGB for ground albedo: " & GroundAlbedo)
    Debug.Print(
        "Sky Gamma (0.1 = 100.0): " & SkyGamma)
    Debug.Print("Minutes of sunlight: " & swModelDocExt.SunLightInformation(swSunlightInfoType_e.swSunlight_LengthOfDay))
    Debug.Print("Sunrise (hours from midnight): " & swModelDocExt.SunLightInformation(swSunlightInfoType_e.swSunlight_Sunrise))

```

```
Debug.Print("Sunset (hours from midnight): " & swModelDocExt.SunLightInformation(swSunlightInfoType_e.swSunlight_Sunset))
'Set sunlight source property values
swMathUtil = swApp.GetMathUtility
nVector(0) = 1 : nVector(1) = 0 : nVector(2) = 0
vVector = nVector
NorthDirection = swMathUtil.CreateVector((vVector))
DateTime =
"11/9/2012 2:48:13 PM"
NorthLatitude = NorthLatitude + 0.1
EastLongitude = EastLongitude + 0.1
TimeZone = TimeZone + 0.5
retval = swModelDocExt.SetSunLightSourcePropertyValues(NorthDirection, NorthLatitude, EastLongitude, TimeZone, DateTime)
End Sub
Public swApp As SldWorks
End
Class
```

Get and Set Table Anchor of Hole Table Example (C#)

This example shows how to get and set the table anchor of a hole table in a drawing.

```
//-----
// Preconditions: Verify that the specified drawing to open exists.
//
// Postconditions:
// 1. Opens the drawing.
// 2. At System.Diagnostics.Debugger.Break(), examine the position
//    of the hole table in the drawing.
// 3. Click the Continue button in the SOLIDWORKS Visual Studio Tools for
//    Applications IDE.
// 4. Sets the position of the hole table's anchor
//    to the specified location.
// 5. Examine the hole table in the drawing.
//
// NOTE: If prompted, do not save changes when closing the drawing.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace TableAnchorPositionCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            string filename = null;
            filename = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\SimpleHole.slddrw";

            ModelDoc2 model = default(ModelDoc2);
            int errors = 0;
            int warnings = 0;
            model = (ModelDoc2)swApp.OpenDoc6(filename, (int)swDocumentTypes_e.swDocDRAWING,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            if (model == null)
                return;

            System.Diagnostics.Debugger.Break();

            TableAnnotation swTable = default(TableAnnotation);

            // If document is a drawing, then continue
            switch (model.GetType())
            {
                case (int)swDocumentTypes_e.swDocDRAWING:
                    DrawingDoc drw = default(DrawingDoc);
                    drw = (DrawingDoc)model;

                    // Get the current sheet
                    Sheet drwSheet = default(Sheet);
                    drwSheet = (Sheet)drw.GetCurrentSheet();

                    // Select the Sheet2 feature
                    ModelDocExtension modelDocExt = default(ModelDocExtension);
                    bool status = false;
                    modelDocExt = (ModelDocExtension)model.Extension;
                    status = modelDocExt.SelectByID2("Sheet2", "SHEET", 0, 0, 0, false, 0, null, 0);

                    // Get the views on Sheet2
                    object[] views = null;
                    views = (object[])drwSheet.GetViews();

                    foreach (object vView in views)
                    {
                        View drwView = default(View);
                        drwView = (View)vView;

                        Feature viewFeature = default(Feature);
                        viewFeature = (Feature)drw.FeatureByName(drwView.Name);

                        // Traverse the features in the view
                        Feature subFeature = default(Feature);
                        subFeature = (Feature)viewFeature.GetFirstSubFeature();

                        // If the feature is HoleTableFeat, then get the table annotations
                        while (!(subFeature == null))
                        {
                            if (subFeature.GetTypeName2() == "HoleTableFeat")
                            {
                                HoleTable swHoleTable = default(HoleTable);
```

```

swHoleTable = (HoleTable)subFeature.GetSpecificFeature2();
object[] holeTables = null;
holeTables = (object[])swHoleTable.GetTableAnnotations();

// If the annotation is a hole table, then continue
if ((holeTables != null))
{
    foreach (object table in holeTables)
    {
        swTable = (TableAnnotation)table;

        // If the hole table is anchored, then continue
        if (swTable.Type == (int)swTableAnnotationType_e.swTableAnnotation_HoleChart)
        {
            if (swTable.Anchored != false)
            {
                TableAnchor holeTableAnchor = default(TableAnchor);
                holeTableAnchor =
(TableAnchor)drwView.Sheet.get_TableAnchor((int)swTableAnnotationType_e.swTableAnnotation_HoleChart);

                // Get the position of the table anchor
                double[] anchorPosition = null;
                anchorPosition = (double[])holeTableAnchor.Position;

                // Determine type of table anchor
                swBOMConfigurationAnchorType_e newCorner = default(swBOMConfigurationAnchorType_e);

                string corner = null;
                switch (swTable.AnchorType)
                {
                    case (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft:
                        corner = " Bottom-left ";
                        newCorner =
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopRight;
                        break;
                    case (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomRight:
                        corner = " Bottom-right ";
                        newCorner =
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft;
                        break;
                    case (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft:
                        corner = " Top-left ";
                        newCorner =
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomRight;
                        break;
                    case (int)swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopRight:
                        corner = " Top-right ";
                        newCorner =
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft;
                        break;
                }

                swTable.AnchorType = (int)newCorner;

                // Set the new position of the table anchor
                double[] dNewPosition = new double[2];
                dNewPosition[0] = 0.0;
                dNewPosition[1] = 0.0;

                holeTableAnchor.Position = dNewPosition;
            }
        }
    }
}

subFeature = (Feature)subFeature.GetNextSubFeature();
}

}

break;
case (int)swDocumentTypes_e.swDocASSEMBLY:
case (int)swDocumentTypes_e.swDocPART:

    break;
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}
}

```

Get and Set Table Anchor of Hole Table Example (VBA)

This example shows how to get and set the table anchor of a hole table in a drawing.

```

'-----
' Preconditions: Verify that the specified drawing to open exists.
'
' Postconditions:
' 1. Opens the drawing.
' 2. At Stop, examine the position of the hole table
'    in the drawing.
' 3. Click the Continue button in the SOLIDWORKS Microsoft Basic
'    for Applications IDE.
' 4. Sets the position of the hole table's anchor
'    to the specified location.
' 5. Examine the hole table in the drawing.
'
' NOTE: If prompted, do not save changes when closing the drawing.
'-----
Option Explicit

Sub main()
    Dim swApp As SldWorks.SldWorks
    Set swApp = Application.SldWorks

    Dim filename As String
    filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SimpleHole.slddrw"

    Dim model As ModelDoc2
    Dim errors As Long
    Dim warnings As Long
    Set model = swApp.OpenDoc6(filename, swDocDRAWING, swOpenDocOptions_Silent, "", errors, warnings)

    If model Is Nothing Then Exit Sub

    Stop

    Dim swTable As TableAnnotation

    ' If document is a drawing, then continue
    Select Case model.GetType
    Case swDocDRAWING
        Dim drw As DrawingDoc
        Set drw = model

        ' Get the current sheet
        Dim drwSheet As Sheet
        Set drwSheet = drw.GetCurrentSheet

        ' Select the Sheet2 feature
        Dim modeldocext As SldWorks.ModelDocExtension
        Dim status As Boolean
        Set modeldocext = model.Extension
        status = modeldocext.SelectByID2("Sheet2", "SHEET", 0, 0, 0, False, 0, Nothing, 0)

        ' Get the views on Sheet2
        Dim views As Variant
        views = drwSheet.GetViews

        Dim vView As Variant
        For Each vView In views
            Dim drwView As View
            Set drwView = vView

            Dim viewFeature As Feature
            Set viewFeature = drw.FeatureByName(drwView.Name)

            ' Traverse the features in the view
            Dim subFeature As Feature
            Set subFeature = viewFeature.GetFirstSubFeature

            ' If the feature is HoleTableFeat, then get the table annotations
            Do Until subFeature Is Nothing
                If subFeature.GetTypeName2 = "HoleTableFeat" Then
                    Dim swHoleTable As HoleTable
                    Set swHoleTable = subFeature.GetSpecificFeature2
                    Dim holeTables
                    holeTables = swHoleTable.GetTableAnnotations

                    ' If the annotation is a hole table, then continue
                    If Not IsEmpty(holeTables) Then
                        Dim table As Variant
                        For Each table In holeTables
                            Set swTable = table

                            ' If the hole table is anchored, then continue
                            If swTable.Type = swTableAnnotation_HoleChart Then
                                If swTable.Anchored <> False Then
                                    Dim holeTableAnchor As TableAnchor
                                    Set holeTableAnchor = drwView.Sheet.TableAnchor(swTable.Annotation_HoleChart)

                                    ' Get the position of the table anchor
                                    Dim anchorPosition
                                    anchorPosition = holeTableAnchor.Position
                                End If
                            End If
                        Next table
                    End If
                End If
            Loop
        Next vView
    End Select
End Sub

```

```

        ' Determine type of table anchor
        Dim newCorner As swBOMConfigurationAnchorType_e

        Dim corner As String
        Select Case swTable.AnchorType
        Case swBOMConfigurationAnchor_BottomLeft
            corner = " Bottom-left "
            newCorner = swBOMConfigurationAnchor_TopRight
        Case swBOMConfigurationAnchor_BottomRight
            corner = " Bottom-right "
            newCorner = swBOMConfigurationAnchor_TopLeft
        Case swBOMConfigurationAnchor_TopLeft
            corner = " Top-left "
            newCorner = swBOMConfigurationAnchor_BottomRight
        Case swBOMConfigurationAnchor_TopRight
            corner = " Top-right "
            newCorner = swBOMConfigurationAnchor_BottomLeft
        End Select

        swTable.AnchorType = newCorner

        ' Set the new position of the table anchor
        Dim dNewPosition(0 To 1) As Double
        dNewPosition(0) = 0#
        dNewPosition(1) = 0#

        holeTableAnchor.Position = dNewPosition
    End If
End If
Next
End If
End If
    Set subFeature = subFeature.GetNextSubFeature
Loop

Next

Case swDocASSEMBLY, swDocPART

End Select

End Sub

```

Get and Set Table Anchor of Hole Table Example (VB.NET)

This example shows how to get and set the table anchor of a hole table in a drawing.

```

'-----
' Preconditions: Verify that the specified drawing to open exists.
'
' Postconditions:
' 1. Opens the drawing.
' 2. At Stop, examine the position of the hole table
'    in the drawing.
' 2. Click the Continue button in the SOLIDWORKS Visual Studio Tools for
'    Applications IDE.
' 4. Sets the position of the hole table's anchor
'    to the specified location.
' 5. Examine the hole table in the drawing.
'
' NOTE: If prompted, do not save changes when closing the drawing.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub Main()
```

```
        Dim filename As String
```

```
        filename = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\SimpleHole.slddrw"
```

```
        Dim model As ModelDoc2
```

```
        Dim errors As Integer
```

```
        Dim warnings As Integer
```

```
        model = swApp.OpenDoc6(filename, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
```

```
        If model Is Nothing Then Exit Sub
```

```
        Stop
```

```
        Dim swTable As TableAnnotation
```

```
        ' If document is a drawing, then continue
```

```
        Select Case model.GetType
```

```
        Case swDocumentTypes_e.swDocDRAWING
```

```
            Dim drw As DrawingDoc
```

```
            drw = model
```

```
            ' Get the current sheet
```

```
            Dim drwSheet As Sheet
```

```
            drwSheet = drw.GetCurrentSheet
```

```
            ' Select the Sheet2 feature
```

```
            Dim modeldocext As ModelDocExtension
```

```
            Dim status As Boolean
```

```
            modeldocext = model.Extension
```

```
            status = modeldocext.SelectByID2("Sheet2", "SHEET", 0, 0, 0, False, 0, Nothing, 0)
```

```
            ' Get the views on Sheet2
```

```
            Dim views As Object
```

```
            views = drwSheet.GetViews
```

```
            Dim vView As Object
```

```
            For Each vView In views
```

```
                Dim drwView As View
```

```
                drwView = vView
```

```
                Dim viewFeature As Feature
```

```
                viewFeature = drw.FeatureByName(drwView.Name)
```

```
                ' Traverse the features in the view
```

```
                Dim subFeature As Feature
```

```
                subFeature = viewFeature.GetFirstSubFeature
```

```
                ' If the feature is HoleTableFeat, then get the table annotations
```

```
                Do Until subFeature Is Nothing
```

```
                    If subFeature.GetTypeName2 = "HoleTableFeat" Then
```

```
                        Dim swHoleTable As HoleTable
```

```
                        swHoleTable = subFeature.GetSpecificFeature2
```

```
                        Dim holeTables As Object
```

```
                        holeTables = swHoleTable.GetTableAnnotations
```

```
                        ' If the annotation is a hole table, then continue
```

```
                        If Not holeTables Is Nothing Then
```

```
                            Dim table As Object
```

```
                            For Each table In holeTables
```

```
                                swTable = table
```

```

        ' If the hole table is anchored, then continue
        If swTable.Type = swTableAnnotationType_e.swTableAnnotation_HoleChart Then
            If swTable.Anchored <> False Then
                Dim holeTableAnchor As TableAnchor
                holeTableAnchor =
drwView.Sheet.TableAnchor(swTableAnnotationType_e.swTableAnnotation_HoleChart)

                ' Get the position of the table anchor
                Dim anchorPosition As Object
                anchorPosition = holeTableAnchor.Position

                ' Determine type of table anchor
                Dim newCorner As swBOMConfigurationAnchorType_e

                Dim corner As String
                Select Case swTable.AnchorType
                    Case swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft
                        corner = " Bottom-left "
                        newCorner = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopRight
                    Case swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomRight
                        corner = " Bottom-right "
                        newCorner = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft
                    Case swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft
                        corner = " Top-left "
                        newCorner = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomRight
                    Case swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopRight
                        corner = " Top-right "
                        newCorner = swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_BottomLeft
                End Select

                swTable.AnchorType = newCorner

                ' Set the new position of the table anchor
                Dim dNewPosition(0 To 1) As Double
                dNewPosition(0) = 0.0#
                dNewPosition(1) = 0.0#

                holeTableAnchor.Position = dNewPosition
            End If
        End If
    Next
End If
End If

    subFeature = subFeature.GetNextSubFeature
Loop

Next

    Case swDocumentTypes_e.swDocASSEMBLY, swDocumentTypes_e.swDocPART

End Select

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get and Set Units Example (C++ COM)

This example shows how to get the current unit settings of the document and change them to inches. Be sure to import swconst.tlb.

The method IModelDoc2::IGetUnits returns an array, so this code must be used in an in-process DLL. Otherwise, use the method IModelDoc2::GetUnits that returns a VARIANT.

```
GetAndSetUnits(ISldWorks* m_pSldWorks)
{
    LPMODELDOC2 pModelDoc = NULL;
    // Retrieve Model Document pointer
    if( S_OK != m_pSldWorks->get_IActiveDoc2( &pModelDoc ) || pModelDoc == NULL )
        return;
    short unitArray[5];
    HRESULT hres = S_OK;
    hres = pModelDoc->IGetUnits(unitArray);
    CString message; // Create message string
    message.Format(_T("Unit Settings are:\n%d\t%d\t%d\t%d\t%d"),
        unitArray[0],unitArray[1],unitArray[2],unitArray[3],unitArray[4]);
    // Send message to the user
    AfxMessageBox (message);
    // If units are not inches, change them to inches
    if (unitArray[0] != swINCHES)
    {
        short denom = 16;
        hres = pModelDoc->SetUnits(swINCHES, swFRACTION, denom, unitArray[3], FALSE);
    }
    pModelDoc->Release();
}
```

Get and Set User Preferences Example (C#)

This sample code demonstrates how to use the methods for getting and setting various system options and document properties.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Create a layer named test in the drawing.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Observe the new settings in the Immediate window.
// 2. Maps the value returned by the GetUserPreferenceInteger
//    method to the corresponding enumerator member in the enumerator
//    online help.
// 3. Click Tools > Options in SOLIDWORKS Design and verify the new settings.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace UserPreferences_CSharp.csproj
{

partial class SolidWorksMacro
{
    public SldWorks swApp;
    public void Main()
    {
        ModelDoc2 Part = null;
        bool boolstatus = false;
        TextFormat swTextFormat = default(TextFormat);
        object TextFormatObj = null;
        ModelDocExtension ModelDocExtension = default(ModelDocExtension);

        Part = (ModelDoc2)swApp.ActiveDoc;
        ModelDocExtension = Part.Extension;

        //The following call demonstrates how to get and set a Tools > Options > System Options > General option

        //Custom property used as component description
        boolstatus =
swApp.SetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription, "Status");
        Debug.Print("Tools > Options > System Options > General > Custom property used as component description: " +
swApp.GetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription));

        //The following calls demonstrate how to get and set Tools > Options > System Options > View options

        //Reverse mouse wheel zoom direction
        swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swViewReverseWheelZoomDirection, true);
        Debug.Print("Tools > Options > System Options > View > Reverse mouse wheel zoom direction: " +
swApp.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swViewReverseWheelZoomDirection));
        //View rotation - Arrow keys
        boolstatus = swApp.SetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewRotationArrowKeys,
0.2268928027593);

        //1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print("Tools > Options > System Options > View > Arrow keys: " +
swApp.GetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewRotationArrowKeys) * 57.3);
        //Convert to degrees
        //View rotation - Mouse speed
        boolstatus = swApp.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed, 56);
        Debug.Print("Tools > Options > System Options > View > Mouse speed: " +
swApp.GetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed));
        //Transitions - View transition
        boolstatus = swApp.SetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewAnimationSpeed, 2.5);
    }
}
```

```

        Debug.Print("Tools > Options > System Options > View > View transition: " +
swApp.GetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewAnimationSpeed));
        //Transitions - Hide/show component
        boolstatus = swApp.SetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent,
0.8000000119209);
        Debug.Print("Tools > Options > System Options > View > Hide/show component: " +
swApp.GetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent));
        //Transitions - Isolate
        boolstatus = swApp.SetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewTransitionIsolate,
1.222222208977);
        Debug.Print("Tools > Options > System Options > View > Isolate: " +
swApp.GetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swViewTransitionIsolate));

        //The following calls demonstrate how to get and set the Tools > Options > Document Properties > Drafting Standard option
        //Maps the value returned by the GetUserPreferenceInteger method to the corresponding enumerator member in the enumerator
online help

        //Overall drafting standard
        boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified, (int)swDetailingStandard_e.swDetailingStandardISO);
        string sText = null;
        sText = "Tools > Options > Document Properties > Drafting Standard > Overall drafting standard is ";
        switch (ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,
(int)swUserPreferenceOption_e.swDetailingNoOptionSpecified))
        {
            case 1:
                Debug.Print(sText + "ANSI");
                break;
            case 2:
                Debug.Print(sText + "ISO");
                break;
            case 3:
                Debug.Print(sText + "DIN");
                break;
            case 4:
                Debug.Print(sText + "JIS");
                break;
            case 5:
                Debug.Print(sText + "BS");
                break;
            case 6:
                Debug.Print(sText + "GOST");
                break;
            case 7:
                Debug.Print(sText + "GB");
                break;
            case 8:
                Debug.Print(sText + "User Defined");
                break;
        }
        //The following calls demonstrate how to get and set Tools > Options > Document Properties > Annotations > Balloons options
        //Maps the value returned by the GetUserPreferenceInteger method to the corresponding enumerator member in the enumerator
online help

        //Leader style - Leader Thickness
        boolstatus =
ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness, 0,
(int)swLineWeights_e.swLW_NUMBER);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Leader Thickness: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness, 0));
        //Leader style - Custom leader thickness
        boolstatus =
ModelDocExtension.SetUserPreferenceDouble((int)swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0,
0.00028);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Custom Leader Thickness: " +
ModelDocExtension.GetUserPreferenceDouble((int)swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0));
        //Frame style - Frame Thickness

```

```

        boolstatus =
ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0,
(int)swLineWeights_e.swLW_NUMBER);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Frame Thickness: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0));
        //Frame style - Custom frame thickness
        boolstatus =
ModelDocExtension.SetUserPreferenceDouble((int)swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0,
0.00028);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Custom Frame Thickness: " +
ModelDocExtension.GetUserPreferenceDouble((int)swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0));
        //Text - Upper - Custom property
        boolstatus =
ModelDocExtension.SetUserPreferenceString((int)swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0, "Source");
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Text Upper Custom property: " +
ModelDocExtension.GetUserPreferenceString((int)swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0));
        //Single balloon - Style
        boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0,
(int)swBalloonStyle_e.swBS_Triangle);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Style: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0));
        //Single balloon - Size
        boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0,
(int)swBalloonFit_e.swBF_3Chars);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Size: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0));
        //Stacked balloons - Style
        boolstatus =
ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle, 0,
(int)swBalloonStyle_e.swBS_Triangle);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Style: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle, 0));
        //Stacked balloons - Size
        boolstatus =
ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0,
(int)swBalloonFit_e.swBF_3Chars);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Size: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0));
        //Auto balloon layout
        boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout,
0, (int)swBalloonLayoutType_e.swDetailingBalloonLayout_Right);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Auto balloon layout: " +
ModelDocExtension.GetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout, 0));
        //Leader display - Use document leader length
        boolstatus =
ModelDocExtension.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength, 0, true);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Leader display - Use document leader length:
" + ModelDocExtension.GetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength, 0));
        //Layer
        boolstatus = ModelDocExtension.SetUserPreferenceString((int)swUserPreferenceStringValue_e.swDetailingLayer,
(int)swUserPreferenceOption_e.swDetailingBalloon, "test");
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Layer: " +
ModelDocExtension.GetUserPreferenceString((int)swUserPreferenceStringValue_e.swDetailingLayer,
(int)swUserPreferenceOption_e.swDetailingBalloon));
        //Font
        TextFormatObj =
ModelDocExtension.GetUserPreferenceTextFormat((int)swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat, 0);
        swTextFormat = (TextFormat)TextFormatObj;
        swTextFormat.Italic = true;
        swTextFormat.Bold = true;
        boolstatus = ModelDocExtension.SetUserPreferenceTextFormat((int)swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat,
0, swTextFormat);
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Font is italic and bold: " + boolstatus);
    }
}
}

```

Get and Set User Preferences Example (VBA)

This sample code demonstrates how to get and set various system options and document properties.

```

'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Create a layer named test in the document.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Observe the new settings in the Immediate window.
' 2. Maps the value returned by the GetUserPreferenceInteger method
'    to the corresponding enumerator member in the enumerator online help.
' 3. Click Tools > Options in SOLIDWORKS Design and verify the new settings.
'-----

Dim swApp As Object
Dim Part As Object
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Dim swTextFormat As SldWorks.TextFormat
Dim TextFormatObj As Object
Dim ModelDocExtension As ModelDocExtension

Sub main()

Set swApp = Application.SldWorks
Set Part = swApp.ActiveDoc
Set ModelDocExtension = Part.Extension

    'The following call demonstrates how to get and set a Tools > Options > System Options > General option
    'Custom property used as component description
    boolstatus = swApp.SetUserPreferenceStringValue(swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription,
"Status")
    Debug.Print "Tools > Options > System Options > General > Custom property used as component description: " &
swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription)

    'The following calls demonstrate how to get and set Tools > Options > System Options > View options
    'Reverse mouse wheel zoom direction
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swViewReverseWheelZoomDirection, True
    Debug.Print "Tools > Options > System Options > View > Reverse mouse wheel zoom direction: " &
swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swViewReverseWheelZoomDirection)

    'View rotation - Arrow keys
    boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewRotationArrowKeys, 0.2268928027593)
    '1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Debug.Print "Tools > Options > System Options > View > Arrow keys: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewRotationArrowKeys) * 57.3 'Convert to degrees

    'View rotation - Mouse speed
    boolstatus = swApp.SetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed, 56)
    Debug.Print "Tools > Options > System Options > View > Mouse speed: " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed)

    'Transitions - View transition
    boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewAnimationSpeed, 2.5)
    Debug.Print "Tools > Options > System Options > View > View transition: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewAnimationSpeed)

    'Transitions - Hide/show component
    boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent,
0.8000000119209)
    Debug.Print "Tools > Options > System Options > View > Hide/show component: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent)

    'Transitions - Isolate

```

```

boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionIsolate, 1.222222208977)
Debug.Print "Tools > Options > System Options > View > Isolate: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionIsolate)

'The following calls demonstrate how to get and set the Tools > Options > Document Properties > Drafting Standard option.
'Map the value returned by the GetUserPreferenceInteger method to the corresponding enumerator member in the enumerator online
help
'Overall drafting standard
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,
swUserPreferenceOption_e.swDetailingNoOptionSpecified, swDetailingStandard_e.swDetailingStandardISO)
Dim sText As String
sText = "Tools > Options > Document Properties > Drafting Standard > Overall drafting standard is "
Select Case ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,
swUserPreferenceOption_e.swDetailingNoOptionSpecified)
Case 1
    Debug.Print sText & "ANSI"
Case 2
    Debug.Print sText & "ISO"
Case 3
    Debug.Print sText & "DIN"
Case 4
    Debug.Print sText & "JIS"
Case 5
    Debug.Print sText & "BS"
Case 6
    Debug.Print sText & "GOST"
Case 7
    Debug.Print sText & "GB"
Case 8
    Debug.Print sText & "User Defined"
End Select

'The following API calls demonstrate how to get and set Tools > Options > Document Properties > Annotations > Balloons options
'Map the value returned by the GetUserPreferenceInteger method to the corresponding enumerator member in the enumerator online
help
'Leader style - Leader Thickness
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness,
0, swLineWeights_e.swLW_NUMBER)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Leader Thickness: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness, 0)

'Leader style - Custom leader thickness
boolstatus =
ModelDocExtension.SetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0, 0.00028)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Custom Leader Thickness: " &
ModelDocExtension.GetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0)

'Frame style - Frame Thickness
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0,
swLineWeights_e.swLW_NUMBER)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Frame Thickness: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0)

'Frame style - Custom frame thickness
boolstatus =
ModelDocExtension.SetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0, 0.00028)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Custom Frame Thickness: " &
ModelDocExtension.GetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0)

'Text - Upper - Custom property
boolstatus = ModelDocExtension.SetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0,
"Source")
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Text Upper Custom property: " &
ModelDocExtension.GetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0)

'Single balloon - Style
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0,
swBalloonStyle_e.swBS_Triangle)

```

```

Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Style: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0)

'Single balloon - Size
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0,
swBalloonFit_e.swBF_3Chars)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Size: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0)

'Stacked balloons - Style
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle, 0,
swBalloonStyle_e.swBS_Triangle)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Style: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle, 0)

'Stacked balloons - Size
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0,
swBalloonFit_e.swBF_3Chars)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Size: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0)

'Auto balloon layout
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout, 0,
swBalloonLayoutType_e.swDetailingBalloonLayout_Right)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Auto balloon layout: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout, 0)

'Leader display - Use document leader length
boolstatus = ModelDocExtension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength, 0,
True)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Leader display - Use document leader length: " &
ModelDocExtension.GetUserPreferenceToggle(swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength, 0)

'Layer
boolstatus = ModelDocExtension.SetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingLayer,
swUserPreferenceOption_e.swDetailingBalloon, "test")
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Layer: " &
ModelDocExtension.GetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingLayer,
swUserPreferenceOption_e.swDetailingBalloon)

'Font...
Set TextFormatObj =
ModelDocExtension.GetUserPreferenceTextFormat(swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat, 0)
Set swTextFormat = TextFormatObj

swTextFormat.Italic = True
swTextFormat.Bold = True

boolstatus = ModelDocExtension.SetUserPreferenceTextFormat(swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat, 0,
swTextFormat)
Debug.Print "Tools > Options > Document Properties > Annotations > Balloons > Font is italic and bold: " & boolstatus
End Sub

```

Get and Set User Preferences Example (VB.NET)

This sample code demonstrates how to get and set various system options and document properties.

```

'-----
' Preconditions:
' 1. Open a drawing document.
' 2. Create a layer named test in the document.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Observe the new settings in the Immediate window.
' 2. Maps the value returned by the GetUserPreferenceInteger method
'    to the corresponding enumerator member in the enumerator online help.
' 3. Click Tools > Options in SOLIDWORKS Design and verify the new settings.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Public Sub main()
        Dim Part As Object
        Dim boolstatus As Boolean
        Dim swTextFormat As TextFormat
        Dim TextFormatObj As Object
        Dim ModelDocExtension As ModelDocExtension

        Part = swApp.ActiveDoc
        ModelDocExtension = Part.Extension

        'The following call demonstrates how to get and set a Tools > Options > System Options > General option

        'Custom property used as component description
        boolstatus = swApp.SetUserPreferenceStringValue(swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription,
"Status")

        Debug.Print("Tools > Options > System Options > General > Custom property used as component description: " &
swApp.GetUserPreferenceStringValue(swUserPreferenceStringValue_e.swCustomPropertyUsedAsComponentDescription))

        'The following calls demonstrate how to get and set Tools > Options > System Options > View options

        'Reverse mouse wheel zoom direction
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swViewReverseWheelZoomDirection, True)
        Debug.Print("Tools > Options > System Options > View > Reverse mouse wheel zoom direction: " &
swApp.GetUserPreferenceToggle(swUserPreferenceToggle_e.swViewReverseWheelZoomDirection))

        'View rotation - Arrow keys
        boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewRotationArrowKeys, 0.2268928027593)
        '1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Debug.Print("Tools > Options > System Options > View > Arrow keys: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewRotationArrowKeys) * 57.3) 'Convert to degrees

        'View rotation - Mouse speed
        boolstatus = swApp.SetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed, 56)
        Debug.Print("Tools > Options > System Options > View > Mouse speed: " &
swApp.GetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swViewRotationMouseSpeed))

        'Transitions - View transition
        boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewAnimationSpeed, 2.5)
        Debug.Print("Tools > Options > System Options > View > View transition: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewAnimationSpeed))

        'Transitions - Hide/show component
        boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent,
0.8000000119209)
        Debug.Print("Tools > Options > System Options > View > Hide/show component: " &
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionHideShowComponent))

        'Transitions - Isolate
        boolstatus = swApp.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionIsolate, 1.222222208977)
    
```

```
Debug.Print("Tools > Options > System Options > View > Isolate: " &  
swApp.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swViewTransitionIsolate))
```

'The following calls demonstrate how to set the Tools > Options > Document Properties > Drafting Standard option using
'SetUserPreferenceInteger

'Maps the value returned by the **GetUserPreferenceInteger** method to the corresponding enumerator member in the enumerator
online help.

'Overall drafting standard

```
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,  
swUserPreferenceOption_e.swDetailingNoOptionSpecified, swDetailingStandard_e.swDetailingStandardISO)
```

```
Dim sText As String
```

```
sText = "Tools > Options > Document Properties > Drafting Standard > Overall drafting standard is "
```

```
Select Case ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingDimensionStandard,  
swUserPreferenceOption_e.swDetailingNoOptionSpecified)
```

```
Case 1
```

```
Debug.Print(sText & "ANSI")
```

```
Case 2
```

```
Debug.Print(sText & "ISO")
```

```
Case 3
```

```
Debug.Print(sText & "DIN")
```

```
Case 4
```

```
Debug.Print(sText & "JIS")
```

```
Case 5
```

```
Debug.Print(sText & "BS")
```

```
Case 6
```

```
Debug.Print(sText & "GOST")
```

```
Case 7
```

```
Debug.Print(sText & "GB")
```

```
Case 8
```

```
Debug.Print(sText & "User Defined")
```

```
End Select
```

'The following calls demonstrate how to set and get Tools > Options > Document Properties > Annotations > Balloons options

'Maps the value returned by the **GetUserPreferenceInteger** method to the corresponding enumerator member in the enumerator
online help

'Leader style - Leader Thickness

```
boolstatus =
```

```
ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness, 0,  
swLineWeights_e.swLW_NUMBER)
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Leader Thickness: " &  
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonLeaderLineThickness, 0))
```

'Leader style - Custom leader thickness

```
boolstatus =
```

```
ModelDocExtension.SetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0, 0.00028)
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Custom Leader Thickness: " &  
ModelDocExtension.GetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonLeaderLineThicknessCustom, 0))
```

'Frame style - Frame Thickness

```
boolstatus =
```

```
ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0,  
swLineWeights_e.swLW_NUMBER)
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Frame Thickness: " &  
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBalloonFrameLineThickness, 0))
```

'Frame style - Custom frame thickness

```
boolstatus =
```

```
ModelDocExtension.SetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0, 0.00028)
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Custom Frame Thickness: " &  
ModelDocExtension.GetUserPreferenceDouble(swUserPreferenceDoubleValue_e.swDetailingBalloonFrameLineThicknessCustom, 0))
```

'Text - Upper - Custom property

```
boolstatus = ModelDocExtension.SetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0,  
"Source")
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Text Upper Custom property: " &  
ModelDocExtension.GetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingBOMUpperCustomProperty, 0))
```

'Single balloon - Style

```
boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0,  
swBalloonStyle_e.swBS_Triangle)
```

```
Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Style: " &  
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonStyle, 0))
```

```

        'Single balloon - Size
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0,
swBalloonFit_e.swBF_3Chars)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Single balloon - Size: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMBalloonFit, 0))
        'Stacked balloons - Style
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle,
0, swBalloonStyle_e.swBS_Triangle)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Style: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonStyle, 0))
        'Stacked balloons - Size
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0,
swBalloonFit_e.swBF_3Chars)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Stacked balloons - Size: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingBOMStackedBalloonFit, 0))
        'Auto balloon layout
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout, 0,
swBalloonLayoutType_e.swDetailingBalloonLayout_Right)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Auto balloon layout: " &
ModelDocExtension.GetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swDetailingAutoBalloonLayout, 0))
        'Leader display - Use document leader length
        boolstatus = ModelDocExtension.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength,
0, True)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Leader display - Use document leader length:
" & ModelDocExtension.GetUserPreferenceToggle(swUserPreferenceToggle_e.swDetailingBalloonUseDocBentLeaderLength, 0))
        'Layer
        boolstatus = ModelDocExtension.SetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingLayer,
swUserPreferenceOption_e.swDetailingBalloon, "test")
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Layer: " &
ModelDocExtension.GetUserPreferenceString(swUserPreferenceStringValue_e.swDetailingLayer,
swUserPreferenceOption_e.swDetailingBalloon))
        'Font...
        TextFormatObj = ModelDocExtension.GetUserPreferenceTextFormat(swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat, 0)
        swTextFormat = TextFormatObj
        swTextFormat.Italic = True
        swTextFormat.Bold = True
        boolstatus = ModelDocExtension.SetUserPreferenceTextFormat(swUserPreferenceTextFormat_e.swDetailingBalloonTextFormat, 0,
swTextFormat)
        Debug.Print("Tools > Options > Document Properties > Annotations > Balloons > Font is italic and bold: " & boolstatus)
    End Sub

    Public swApp As SldWorks
End Class

```

Get and Set Document Units Example (C#)

This example shows how to get and set document angle units.

```
//-----
// Preconditions:
// 1. Open a part document.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Observe the document's angle unit properties in the Immediate window.
// 2. Modifies the decimal places and type of the angle unit.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetSetUserUnits_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        UserUnit docUserUnit;
        bool boolstatus;
        ModelDocExtension ModelDocExtension;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            ModelDocExtension = Part.Extension;
            docUserUnit = (UserUnit)Part.GetUserUnit((int)swUserUnitsType_e.swAngleUnit);

            //Get document's current angle unit properties
            Debug.Print("Angle unit properties: ");
            Debug.Print(" Display leading zero? " + docUserUnit.DisplayLeadingZero);
            Debug.Print(" Fraction display as defined in swFractionDisplay_e: " + docUserUnit.FractionBase);
            Debug.Print(" Fraction denominator: " + docUserUnit.FractionValue);
            Debug.Print(" Pad with zeroes? " + docUserUnit.PadZero);
            Debug.Print(" Round to fraction? " + docUserUnit.RoundToFraction);
            Debug.Print(" Decimal separator character: " + docUserUnit.SeparatorCharacter);
            Debug.Print(" Decimal places: " + docUserUnit.SignificantDigits);
            Debug.Print(" Units as defined in swAngleUnit_e: " + docUserUnit.SpecificUnitType);
            Debug.Print(" Unit type as defined in swUserUnitsType_e: " + docUserUnit.UnitType);
            Debug.Print(" Units are metric? " + docUserUnit.IsMetric());
            Debug.Print(" Conversion factor: " + docUserUnit.GetConversionFactor());
            Debug.Print(" Full units name: " + docUserUnit.GetFullUnitName(true));
            Debug.Print(" Units name: " + docUserUnit.GetUnitsString(true));
            Debug.Print(" Angle tolerance: " + docUserUnit.GetUserAngleTolerance());
            Debug.Print("");

            double sysVal = 0;
            sysVal = docUserUnit.ConvertDoubleToSystemValue(2.0);
            Debug.Print(" 2.0 in document angle units: " + sysVal + ".");

            double compVal = 0;
            boolstatus = docUserUnit.ConvertToSystemValue("10 / 2", ref compVal);
            Debug.Print(" 10 / 2 in document angle units: " + compVal + ".");

            string stringVal = null;
            stringVal = docUserUnit.ConvertToUserUnit(2.0, true, true);
            Debug.Print(" 2.0 in complete document angle units: " + stringVal + ".");

            //Modify decimal places and type of angle unit
            boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swUnitsAngular, (int)swUserF
            boolstatus = ModelDocExtension.SetUserPreferenceInteger((int)swUserPreferenceIntegerValue_e.swUnitsAngularDecimalPlaces,

        }

        public SldWorks swApp;
    }
}
```

Get and Set Document Units Example (VBA)

This example shows how to get and set document angle units.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Observe the document's angle unit properties in the Immediate window.
' 2. Modifies the decimal places and type of the angle unit.
'-----

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim docUserUnit As SldWorks.UserUnit
Dim boolstatus As Boolean
Dim ModelDocExtension As SldWorks.ModelDocExtension

Option Explicit

Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set ModelDocExtension = Part.Extension
    Set docUserUnit = Part.GetUserUnit(swAngleUnit)

    'Get document's current angle unit properties
    Debug.Print "Angle unit properties: "
    Debug.Print " Display leading zero? " & docUserUnit.DisplayLeadingZero
    Debug.Print " Fraction display as defined in swFractionDisplay_e: " & docUserUnit.FractionBase
    Debug.Print " Fraction denominator: " & docUserUnit.FractionValue
    Debug.Print " Pad with zeroes? " & docUserUnit.PadZero
    Debug.Print " Round to fraction? " & docUserUnit.RoundToFraction
    Debug.Print " Decimal separator character: " & docUserUnit.SeparatorCharacter
    Debug.Print " Decimal places: " & docUserUnit.SignificantDigits
    Debug.Print " Units as defined in swAngleUnit_e: " & docUserUnit.SpecificUnitType
    Debug.Print " Unit type as defined in swUserUnitsType_e: " & docUserUnit.UnitType
    Debug.Print " Units are metric? " & docUserUnit.IsMetric
    Debug.Print " Conversion factor: " & docUserUnit.GetConversionFactor
    Debug.Print " Full units name: " & docUserUnit.GetFullUnitName(True)
    Debug.Print " Units name: " & docUserUnit.GetUnitsString(True)
    Debug.Print " Angle tolerance: " & docUserUnit.GetUserAngleTolerance
    Debug.Print ""

    Dim sysVal As Double
    sysVal = docUserUnit.ConvertDoubleToSystemValue(2#)
    Debug.Print " 2.0 in document angle units: " & sysVal & "."

    Dim compVal As Double
    boolstatus = docUserUnit.ConvertToSystemValue("10 / 2", compVal)
    Debug.Print " 10 / 2 in document angle units: " & compVal & "."

    Dim stringVal As String
    stringVal = docUserUnit.ConvertToUserUnit(2#, True, True)
    Debug.Print " 2.0 in complete document angle units: " & stringVal & "."

    'Modify decimal places and type of angle unit
    boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swUnitsAngular,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, swAngleUnit_e.swDEG_MIN_SEC)
    boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swUnitsAngularDecimalPlaces,
    swUserPreferenceOption_e.swDetailingNoOptionSpecified, 2)

End Sub

```

Get and Set Document Units Example (VB.NET)

This example shows how to get and set document angle units.

```

'-----
' Preconditions:
' 1. Open a part document.
' 2. Open an Immediate window.
'-----
' Postconditions:
' 1. Observe the document's angle unit properties in the Immediate window.
' 2. Modifies the decimal places and type of the angle unit.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim docUserUnit As UserUnit
    Dim boolstatus As Boolean
    Dim ModelDocExtension As ModelDocExtension

    Sub main()

        Part = swApp.ActiveDoc
        ModelDocExtension = Part.Extension
        docUserUnit = Part.GetUserUnit(swUserUnitsType_e.swAngleUnit)

        'Get document's current angle unit properties
        Debug.Print("Angle unit properties: ")
        Debug.Print("  Display leading zero? " & docUserUnit.DisplayLeadingZero)
        Debug.Print("  Fraction display as defined in swFractionDisplay_e: " & docUserUnit.FractionBase)
        Debug.Print("  Fraction denominator: " & docUserUnit.FractionValue)
        Debug.Print("  Pad with zeroes? " & docUserUnit.PadZero)
        Debug.Print("  Round to fraction? " & docUserUnit.RoundToFraction)
        Debug.Print("  Decimal separator character: " & docUserUnit.SeparatorCharacter)
        Debug.Print("  Decimal places: " & docUserUnit.SignificantDigits)
        Debug.Print("  Units as defined in swAngleUnit_e: " & docUserUnit.SpecificUnitType)
        Debug.Print("  Unit type as defined in swUserUnitsType_e: " & docUserUnit.UnitType)
        Debug.Print("  Units are metric? " & docUserUnit.IsMetric)
        Debug.Print("  Conversion factor: " & docUserUnit.GetConversionFactor)
        Debug.Print("  Full units name: " & docUserUnit.GetFullUnitName(True))
        Debug.Print("  Units name: " & docUserUnit.GetUnitsString(True))
        Debug.Print("  Angle tolerance: " & docUserUnit.GetUserAngleTolerance)
        Debug.Print("")

        Dim sysVal As Double
        sysVal = docUserUnit.ConvertDoubleToSystemValue(2.0#)
        Debug.Print("  2.0 in document angle units: " & sysVal & ".")

        Dim compVal As Double
        boolstatus = docUserUnit.ConvertToSystemValue("10 / 2", compVal)
        Debug.Print("  10 / 2 in document angle units: " & compVal & ".")

        Dim stringVal As String
        stringVal = docUserUnit.ConvertToUserUnit(2.0#, True, True)
        Debug.Print("  2.0 in complete document angle units: " & stringVal & ".")

        'Modify decimal places and type of angle unit
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swUnitsAngular, swUserPreferenceOptic
        boolstatus = ModelDocExtension.SetUserPreferenceInteger(swUserPreferenceIntegerValue_e.swUnitsAngularDecimalPlaces, swUserPr

    End Sub

    Public swApp As SldWorks

End Class

```

Get and Set Whether to Hide Cutting Line Shoulders Example (C#)

This example shows how to get and set whether to hide cutting line shoulders in a section view.

```
//-----
// Preconditions:
// 1. Verify that the part and templates exist.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Creates a drawing of the part.
// 3. Creates a section view.
// 4. Gets and sets whether to hide cutting line shoulders in the section
//    view.
// 5. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save it or the drawing.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDrawing = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            View swView = default(View);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchMgr = default(SketchManager);
            DrSection swSectionView = default(DrSection);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            double swSheetWidth = 0;
            double swSheetHeight = 0;
            string drawingTemplate = null;
            string sheetTemplate = null;

            //Open part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cam roller.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Create drawing of part
            swSheetWidth = 1.189;
            swSheetHeight = 0.841;
            drawingTemplate = "C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\Drawing.drwdot";
            swDrawing = (DrawingDoc)swApp.NewDocument(drawingTemplate, (int)swDwgPaperSizes_e.swDwgPapersUserDefined, swSheetWidth, swSheetHeight);
            swSheet = (Sheet)swDrawing.GetCurrentSheet();
            swSheet.SetProperties2((int)swDwgPaperSizes_e.swDwgPapersUserDefined, (int)swDwgTemplates_e.swDwgTemplateCustom, 1, 2, false, swSheetWidth, sw
SheetHeight, true);
            sheetTemplate = "C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\lang\\english\\sheetformat\\a0 - iso.slddrt";
            swSheet.SetTemplateName(sheetTemplate);
            swSheet.ReloadTemplate(true);
            status = swDrawing.GenerateViewPaletteViews(fileName);
            swView = (View)swDrawing.DropDrawingViewFromPalette2("Left", 0.580930433566434, 0.431525272727273, 0);

            //Create section view
            swDrawing = (DrawingDoc)swApp.ActiveDoc;
            status = swDrawing.ActivateView("Drawing View1");
            swModel.ClearSelection2(true);
            swModel = (ModelDoc2)swDrawing;
            swSketchMgr = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.0, 0.0, 0.0, 0.012168, 0.021283, 0.0);
            swSketchSegment = (SketchSegment)swSketchMgr.CreateLine(0.0, 0.0, 0.0, 0.024347, -0.010966, 0.0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0.690604633175108, 0.625483883858213, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0.747211061353527, 0.357889859742052, 0, true, 0, null, 0);
            swView = (View)swDrawing.CreateSectionViewAt5(0.676815388637685, 0.116110180826413, 0, "A", (int)swCreateSectionViewAtOptions_e.swCreateSectio
nView_OffsetSection, null, 0);
            status = swDrawing.ActivateView("Drawing View2");
            swModel.ClearSelection2(true);

            //Get section view and get and set whether to hide cutting line shoulders
            swSectionView = (DrSection)swView.GetSection();
            if (swSectionView.CuttingLineShoulders)
            {
                Debug.Print("Hide cutting line shoulders = True");
                Debug.Print("Setting hide cutting line shoulders to False");
                swSectionView.CuttingLineShoulders = false;
                Debug.Print(" Hide cutting line shoulders = " + swSectionView.CuttingLineShoulders);
            }
            else
            {
                Debug.Print("Hide cutting line shoulders = False");
                Debug.Print("Setting hide cutting line shoulders to True");
                swSectionView.CuttingLineShoulders = true;
                Debug.Print(" Hide cutting line shoulders = " + swSectionView.CuttingLineShoulders);
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Get and Set Whether to Hide Cutting Line Shoulders Example (VBA)

This example shows how to get and set whether to hide cutting line shoulders in a section view.

```

'-----
' Preconditions:
' 1. Verify that the part and templates exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Creates a drawing of the part.
' 3. Creates a section view.
' 4. Gets and sets whether to hide cutting line shoulders in the section
'    view.
' 5. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save it or the drawing.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swSheet As SldWorks.Sheet
Dim swView As SldWorks.View
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketchMgr As SldWorks.SketchManager
Dim swSectionView As SldWorks.DrSection
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim swSheetWidth As Double
Dim swSheetHeight As Double
Dim drawingTemplate As String
Dim sheetTemplate As String

Sub main()

    Set swApp = Application.SldWorks

    'Open part
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cam roller.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Create drawing of part
    swSheetWidth = 1.189
    swSheetHeight = 0.841
    drawingTemplate = "C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Drawing.drwdot"
    Set swDrawing = swApp.NewDocument(drawingTemplate, swDwgPaperSizes_e.swDwgPapersUserDefined, swSheetWidth, swSheetHeight)
    Set swSheet = swDrawing.GetCurrentSheet()
    swSheet.SetProperties2 swDwgPaperSizes_e.swDwgPapersUserDefined, swDwgTemplates_e.swDwgTemplateCustom, 1, 2, False, swSheetWidth, swSheetHeight, True
    sheetTemplate = "C:\ProgramData\SolidWorks\SOLIDWORKS 2017\lang\english\sheetformat\A0 - iso.sldprt"
    swSheet.SetTemplateName sheetTemplate
    swSheet.ReloadTemplate True
    status = swDrawing.GenerateViewPaletteViews(fileName)
    Set swView = swDrawing.DropDrawingViewFromPalette2("Left", 0.580930433566434, 0.431525272727273, 0)

    'Create section view
    Set swDrawing = swApp.ActiveDoc
    status = swDrawing.ActiveView("Drawing View1")
    swModel.ClearSelection2 True
    Set swModel = swDrawing
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateLine(0#, 0#, 0#, 0.012168, 0.021283, 0#)
    Set swSketchSegment = swSketchMgr.CreateLine(0#, 0#, 0#, 0.024347, -0.010966, 0#)
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0.690604633175108, 0.625483883858213, 0, False, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0.747211061353527, 0.357889859742052, 0, True, 0, Nothing, 0)
    Set swView = swDrawing.CreateSectionViewAt5(0.676815388637685, 0.116110180826413, 0, "A",
    swCreateSectionViewAtOptions_e.swCreateSectionView_OffsetSection, Nothing, 0)
    status = swDrawing.ActiveView("Drawing View2")
    swModel.ClearSelection2 True

    'Get section view and get and set whether to hide cutting line shoulders
    Set swSectionView = swView.GetSection
    If swSectionView.CuttingLineShoulders Then
        Debug.Print "Hide cutting line shoulders = True"
        Debug.Print "Setting hide cutting line shoulders to False"
        swSectionView.CuttingLineShoulders = False
        Debug.Print " Hide cutting line shoulders = " & swSectionView.CuttingLineShoulders
    Else
        Debug.Print "Hide cutting line shoulders = False"
        Debug.Print "Setting hide cutting line shoulders to True"
        swSectionView.CuttingLineShoulders = True
        Debug.Print " Hide cutting line shoulders = " & swSectionView.CuttingLineShoulders
    End If

End Sub

```

Get and Set Whether to Hide Cutting Line Shoulders Example (VB.NET)

This example shows how to get and set whether to hide cutting line shoulders in a section view.

```

'-----
' Preconditions:
' 1. Verify that the part and templates exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Creates a drawing of the part.
' 3. Creates a section view.
' 4. Gets and sets whether to hide cutting line shoulders in the section
'    view.
' 5. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save it or the drawing.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swDrawing As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchSegment As SketchSegment
        Dim swSketchMgr As SketchManager
        Dim swSectionView As DrSection
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim swSheetWidth As Double
        Dim swSheetHeight As Double
        Dim drawingTemplate As String
        Dim sheetTemplate As String

        'Open part
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cam roller.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Create drawing of part
        swSheetWidth = 1.189
        swSheetHeight = 0.841
        drawingTemplate = "C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Drawing.drwdot"
        swDrawing = swApp.NewDocument(drawingTemplate, swDwgPaperSizes_e.swDwgPapersUserDefined, swSheetWidth, swSheetHeight)
        swSheet = swDrawing.GetCurrentSheet()
        swSheet.SetProperties2(swDwgPaperSizes_e.swDwgPapersUserDefined, swDwgTemplates_e.swDwgTemplateCustom, 1, 2, False, swSheetWidth, swSheetHeight, True)

        sheetTemplate = "C:\ProgramData\SolidWorks\SOLIDWORKS 2017\lang\english\sheetformat\iso - iso.slddrt"
        swSheet.SetTemplateName(sheetTemplate)
        swSheet.ReloadTemplate(True)
        status = swDrawing.GenerateViewPaletteViews(fileName)
        swView = swDrawing.DropDrawingViewFromPalette2("Left", 0.580930433566434, 0.431525272727273, 0)

        'Create section view
        swDrawing = swApp.ActiveDoc
        status = swDrawing.ActivateView("Drawing View1")
        swModel.ClearSelection2(True)
        swModel = swDrawing
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateLine(0.0#, 0.0#, 0.0#, 0.012168, 0.021283, 0.0#)
        swSketchSegment = swSketchMgr.CreateLine(0.0#, 0.0#, 0.0#, 0.024347, -0.010966, 0.0#)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Line1", "SKETCHSEGMENT", 0.690604633175108, 0.625483883858213, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("Line2", "SKETCHSEGMENT", 0.747211061353527, 0.357889859742052, 0, True, 0, Nothing, 0)
        swView = swDrawing.CreateSectionViewAt5(0.676815388637685, 0.116110180826413, 0, "A", swCreateSectionViewAtOptions_e.swCreateSectionView_OffsetSection, Nothing, 0)
        status = swDrawing.ActivateView("Drawing View2")
        swModel.ClearSelection2(True)

        'Get section view and get and set whether to hide cutting line shoulders
        swSectionView = swView.GetSection
        If swSectionView.CuttingLineShoulders Then
            Debug.Print("Hide cutting line shoulders = True")
            Debug.Print("Setting hide cutting line shoulders to False")
            swSectionView.CuttingLineShoulders = False
            Debug.Print(" Hide cutting line shoulders = " & swSectionView.CuttingLineShoulders)
        Else
            Debug.Print("Hide cutting line shoulders = False")
            Debug.Print("Setting hide cutting line shoulders to True")
            swSectionView.CuttingLineShoulders = True
            Debug.Print(" Hide cutting line shoulders = " & swSectionView.CuttingLineShoulders)
        End If

    End Sub

End Class

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Get Angle Between Plane and Line Example (VBA)

This example shows how to get the angle between a selected plane and a selected sketch line.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates a 3D sketch.
' 3. Selects the Top Plane and a sketch line in the
'    3D sketch.
' 4. Gets the normal vector, curve vector, and the angle
'    between the selected plane and sketch line.
' 5. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swMath As SldWorks.MathUtility
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketch As SldWorks.Sketch
Dim swFeature As SldWorks.Feature
Dim swRefPlane As SldWorks.RefPlane
Dim normVec As MathVector
Dim curveVec As SldWorks.MathVector
Dim swCurve As SldWorks.Curve
Dim dirArr(2) As Double
Dim params As Variant
Dim crossVec As SldWorks.MathVector
Dim dot As Double
Dim vecLen As Double
Dim angle As Double
Dim boolstatus As Boolean
Dim longstatus As Long

Sub main()
    Set swApp = Application.SldWorks

    longstatus = swApp.ResetUntitledCount(0, 0, 0)
    Set swModel = swApp.NewDocument("C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Part.prtdot", 0, 0, 0)
    swApp.ActivateDoc2 "Part1", False, longstatus
    Set swModel = swApp.ActiveDoc

    ' Insert 3D sketch
    Set swSketchManager = swModel.SketchManager
    swSketchManager.Insert3DSketch True
    Set swSketchSegment = swSketchManager.CreateLine(-0.038076, 0.043671, -0#, -0.01322, 0.054563, -0#)
    Set swSketch = swModel.GetActiveSketch2()
    boolstatus = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0)
    Set swSketchSegment = swSketchManager.CreateLine(-0.01322, 0.054563, -0#, -0.01322, 0.08124, 0.018547)
    Set swSketch = swModel.GetActiveSketch2()
    boolstatus = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0)
    Set swSketchSegment = swSketchManager.CreateLine(-0.01322, 0.08124, 0.018547, 0.000568, 0.08124, 0.004759)
    Set swSketch = swSketchManager.ActiveSketch
    boolstatus = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    ' Select Top Plane and a line in the 3D sketch
    Set swMath = swApp.GetMathUtility
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    swModel.ClearSelection2 True
    boolstatus = swModelDocExt.SelectByID2("Top Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    boolstatus = swModelDocExt.SelectByID2("Line1@3DSketch1", "EXTSKETCHSEGMENT", -3.42695618142891E-02, 4.53389966494514E-02, 0, True, 0, Nothing, 0)
    Set swSketchSegment = swSelMgr.GetSelectedObject6(2, -1)
    Set swRefPlane = swFeature.GetSpecificFeature2

    ' Get the normal and curve vectors
    dirArr(0) = 0#
    dirArr(1) = 0#
    dirArr(2) = 1#

    Set normVec = swMath.CreateVector((dirArr))
    Set normVec = normVec.MultiplyTransform(swRefPlane.Transform)
    Debug.Print "Normal vector: " & normVec.ArrayData(0), normVec.ArrayData(1), normVec.ArrayData(2)

    Set swCurve = swSketchSegment.GetCurve
    params = swCurve.LineParams
    dirArr(0) = params(3)
    dirArr(1) = params(4)
    dirArr(2) = params(5)
    Set curveVec = swMath.CreateVector((dirArr))
    Debug.Print "Curve vector: " & curveVec.ArrayData(0), curveVec.ArrayData(1), curveVec.ArrayData(2)

    Set crossVec = curveVec.Cross(normVec)

    ' Get the angle between the Top Plane and
    ' selected line in the 3D sketch
```

```
dot = curveVec.dot(normVec)
vecLen = crossVec.GetLength()
angle = Atn(dot / vecLen)
Debug.Print "Angle:      " & angle
```

```
End Sub
```

Get Angle of Hole Not Normal to a Face Example (VBA)

This example shows how to get the angle of a hole not normal to a face.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cover_datum.sldprt.
' 2. Select the edge of a hole that enters the part on a flat face.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Click Continue at each Debug.Assert statement.
' 2. Gets an angle in the first quadrant, which should be
'    the angle measured between the hole axis and face
'    normal.
' 3. Examine the Immediate window.
'-----
Option Explicit

Const PI As Double = 3.14159265358979

Function Arccos(X As Double) As Double
    If Abs(1# - X) < 0.00000001 Then Arccos = PI / 2#: Exit Function
    Arccos = Atn(-X / Sqr(-X * X + 1)) + 2 * Atn(1)
End Function

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swMathUtil As SldWorks.MathUtility
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEdge As SldWorks.Edge
    Dim vFace As Variant
    Dim swFace(1) As SldWorks.Face2
    Dim swSurf(1) As SldWorks.Surface
    Dim swCylSurf As SldWorks.Surface
    Dim swFlatSurf As SldWorks.Surface
    Dim vCylinder As Variant
    Dim vPlane As Variant
    Dim nVector(2) As Double
    Dim vVector As Variant
    Dim i As Long
    Dim swCylAxis As SldWorks.MathVector
    Dim swFlatNorm As SldWorks.MathVector
    Dim nDotProd As Double
    Dim nAngle As Double
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swMathUtil = swApp.GetMathUtility
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swEdge = swSelMgr.GetSelectedObject6(1, -1)

    vFace = swEdge.GetTwoAdjacentFaces2
    Set swFace(0) = vFace(0)
    Set swFace(1) = vFace(1)
    Set swSurf(0) = swFace(0).GetSurface
    Set swSurf(1) = swFace(1).GetSurface

    If swSurf(0).IsCylinder Then
        Set swCylSurf = swSurf(0)
        Set swFlatSurf = swSurf(1)
    Else
        Debug.Assert swSurf(0).IsPlane
        Debug.Assert swSurf(1).IsCylinder
        Set swCylSurf = swSurf(1)
        Set swFlatSurf = swSurf(0)
    End If

    vCylinder = swCylSurf.CylinderParams
    vPlane = swFlatSurf.PlaneParams

    nVector(0) = vCylinder(3): nVector(1) = vCylinder(4): nVector(2) = vCylinder(5)
    vVector = nVector
    Set swCylAxis = swMathUtil.CreateVector((vVector))
    nVector(0) = vPlane(0): nVector(1) = vPlane(1): nVector(2) = vPlane(2)
    vVector = nVector
    Set swFlatNorm = swMathUtil.CreateVector((vVector))
    nDotProd = swCylAxis.Dot(swFlatNorm)

    ' Ensure angle is in first quadrant
    nAngle = Arccos(Abs(nDotProd)) * 180# / PI

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " cylinder origin = (" & vCylinder(0) * 1000# & ", " & vCylinder(1) * 1000# & ", " & vCylinder(2) * 1000# & ") mm"
    Debug.Print " cylinder axis = (" & vCylinder(3) & ", " & vCylinder(4) & ", " & vCylinder(5) & ")"
    Debug.Print " cylinder radius = " & vCylinder(6) * 1000# & " mm"
    Debug.Print " flat normal = (" & vPlane(0) & ", " & vPlane(1) & ", " & vPlane(2) & ")"
    Debug.Print " flat root = (" & vPlane(3) * 1000# & ", " & vPlane(4) * 1000# & ", " & vPlane(5) * 1000# & ") mm"
    Debug.Print " angle = " & nAngle & " deg"

End Sub
```

Get Annotation Object Example (C#)

This example shows how to obtain the annotation object for the selected annotation symbol.

```
//-----
// Preconditions:
// 1. Open a model document that contains an annotation symbol.
// 2. Select the annotation symbol.
//
// Postconditions: The annotation object for the selected symbol is obtained.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetAnnotationObject_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr SelMgr;
        object AnnoObject;
        WeldSymbol aWeldSymbol;
        WeldBead aWeldBead;
        TableAnnotation aTableAnno;
        SFSymbol anSFSymbol;
        RevisionCloud aRevisionCloud;
        Note aNote;
        MultiJogLeader aLeader;
        Gtol aGtol;
        DowelSymbol aDowelSym;
        DisplayDimension aDispDim;
        DatumTargetSym aDatTarSym;
        DatumTag aDatumTag;
        DatumOrigin aDatumOrigin;
        CustomSymbol aCustomSymbol;
        CenterMark aCenterMark;
        CenterLine aCenterLine;
        CThread aCThread;

        Annotation Anno;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)Part.SelectionManager;
            AnnoObject = SelMgr.GetSelectedObject6(1, -1);

            switch (SelMgr.GetSelectedObjectType3(1, -1))
            {
                case (int)swSelectType_e.swSelWELDS:
                    Debug.Print("Selected annotation is a WeldSymbol.");
                    aWeldSymbol = (WeldSymbol)AnnoObject;
                    Anno = (Annotation)aWeldSymbol.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelWELDBEADS:
                    Debug.Print("Selected annotation is a WeldBead.");
                    aWeldBead = (WeldBead)AnnoObject;
                    Anno = (Annotation)aWeldBead.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelANNOTATIONTABLES:
                    Debug.Print("Selected annotation is a TableAnnotation.");
                    aTableAnno = (TableAnnotation)AnnoObject;
                    Anno = (Annotation)aTableAnno.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelSFSYMBOLS:
                    Debug.Print("Selected annotation is an SFSymbol.");
                    anSFSymbol = (SFSymbol)AnnoObject;
                    Anno = (Annotation)anSFSymbol.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelREVISIONCLOUDS:
                    Debug.Print("Selected annotation is a RevisionCloud.");
                    aRevisionCloud = (RevisionCloud)AnnoObject;
                    Anno = (Annotation)aRevisionCloud.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelNOTES:
                    Debug.Print("Selected annotation is a Note.");
                    aNote = (Note)AnnoObject;
                    Anno = (Annotation)aNote.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelLEADERS:
                    Debug.Print("Selected annotation is a Leader.");
                    aLeader = (MultiJogLeader)AnnoObject;
                    Anno = (Annotation)aLeader.GetAnnotation();
                    break;
                case (int)swSelectType_e.swSelGTOLS:

```

```

        Debug.Print("Selected annotation is a Gtol.");
        aGtol = (Gtol)AnnoObject;
        Anno = (Annotation)aGtol.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelDOWELSYMS:
        Debug.Print("Selected annotation is a DowelSymbol.");
        aDowelSym = (DowelSymbol)AnnoObject;
        Anno = (Annotation)aDowelSym.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelDIMENSIONS:
        Debug.Print("Selected annotation is a DisplayDimension.");
        aDispDim = (DisplayDimension)AnnoObject;
        Anno = (Annotation)aDispDim.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelDTMTARGS:
        Debug.Print("Selected annotation is a DatumTargetSym.");
        aDatTarSym = (DatumTargetSym)AnnoObject;
        Anno = (Annotation)aDatTarSym.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelDATUMTAGS:
        Debug.Print("Selected annotation is a DatumTag.");
        aDatumTag = (DatumTag)AnnoObject;
        Anno = (Annotation)aDatumTag.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelDATUMPOINTS:
        Debug.Print("Selected annotation is a DatumOrigin.");
        aDatumOrigin = (DatumOrigin)AnnoObject;
        Anno = (Annotation)aDatumOrigin.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelCUSTOMSYMBOLS:
        Debug.Print("Selected annotation is a CustomSymbol.");
        aCustomSymbol = (CustomSymbol)AnnoObject;
        Anno = (Annotation)aCustomSymbol.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelCTHREADS:
        Debug.Print("Selected annotation is a CThread.");
        aCThread = (CThread)AnnoObject;
        Anno = (Annotation)aCThread.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelCENTERMARKSYMS:
        Debug.Print("Selected annotation is a CenterMark.");
        aCenterMark = (CenterMark)AnnoObject;
        Anno = (Annotation)aCenterMark.GetAnnotation();
        break;
    case (int)swSelectType_e.swSelCENTERLINES:
        Debug.Print("Selected annotation is a CenterLine.");
        aCenterLine = (CenterLine)AnnoObject;
        Anno = (Annotation)aCenterLine.GetAnnotation();
        break;
}

Debug.Print("Annotation: " + Anno.GetName());
}

```

```

public SldWorks swApp;

```

```

}

```

```

}

```

Get Annotation Object Example (VBA)

This example shows how to obtain the annotation object for the selected annotation symbol.

```
'-----
' Preconditions:
' 1. Open a model document that contains an annotation symbol.
' 2. Select the annotation symbol.
'
' Postconditions: The annotation object for the selected symbol is obtained.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim SelMgr As SldWorks.SelectionMgr
Dim AnnoObject As Object
Dim aWeldSymbol As SldWorks.WeldSymbol
Dim aWeldBead As SldWorks.WeldBead
Dim aView As SldWorks.View
Dim aTableAnno As SldWorks.TableAnnotation
Dim anSFSymbol As SldWorks.SFSymbol
Dim aRevisionCloud As SldWorks.RevisionCloud
Dim aNote As SldWorks.Note
Dim aLeader As SldWorks.MultijogLeader
Dim aGtol As SldWorks.Gtol
Dim aDowelSym As SldWorks.DowelSymbol
Dim aDispDim As SldWorks.DisplayDimension
Dim aDatTarSym As SldWorks.DatumTargetSym
Dim aDatumTag As SldWorks.DatumTag
Dim aDatumOrigin As SldWorks.DatumOrigin
Dim aCustomSymbol As SldWorks.CustomSymbol
Dim aCenterMark As SldWorks.CenterMark
Dim aCenterLine As SldWorks.CenterLine
Dim aCThread as SldWorks.CThread
Dim Anno As SldWorks.Annotation

Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set SelMgr = Part.SelectionManager
    Set AnnoObject = SelMgr.GetSelectedObject6(1, -1)

    Select Case SelMgr.GetSelectedObjectType3(1, -1)
        Case swSelWELDS:
            Debug.Print "Selected annotation is a WeldSymbol."
            Set aWeldSymbol = AnnoObject
            Set Anno = aWeldSymbol.GetAnnotation
        Case swSelWELDBEADS:
            Debug.Print "Selected annotation is a WeldBead."
            Set aWeldBead = AnnoObject
            Set Anno = aWeldBead.GetAnnotation
        Case swSelDRAWINGVIEWS:
            Debug.Print "Selected annotation is a View."
            Set aView = AnnoObject
            Set Anno = aView.GetAnnotation
        Case swSelANNOTATIONTABLES:
            Debug.Print "Selected annotation is a TableAnnotation."
            Set aTableAnno = AnnoObject
            Set Anno = aTableAnno.GetAnnotation
        Case swSelSFSYMBOLS:
            Debug.Print "Selected annotation is an SFSymbol."
            Set anSFSymbol = AnnoObject
            Set Anno = anSFSymbol.GetAnnotation
        Case swSelREVISIONCLOUDS:
            Debug.Print "Selected annotation is a RevisionCloud."
            Set aRevisionCloud = AnnoObject
            Set Anno = aRevisionCloud.GetAnnotation
        Case swSelNOTES:
            Debug.Print "Selected annotation is a Note."
            Set aNote = AnnoObject
            Set Anno = aNote.GetAnnotation
        Case swSelLEADERS:
            Debug.Print "Selected annotation is a Leader."
            Set aLeader = AnnoObject
            Set Anno = aLeader.GetAnnotation
        Case swSelGTOLS:
            Debug.Print "Selected annotation is a Gtol."
            Set aGtol = AnnoObject
            Set Anno = aGtol.GetAnnotation
        Case swSelDOWELSYMS:
            Debug.Print "Selected annotation is a DowelSymbol."
            Set aDowelSym = AnnoObject
            Set Anno = aDowelSym.GetAnnotation
        Case swSelDIMENSIONS:
            Debug.Print "Selected annotation is a DisplayDimension."
            Set aDispDim = AnnoObject
            Set Anno = aDispDim.GetAnnotation
        Case swSelDTMTARGS
```

```

        Debug.Print "Selected annotation is a DatumTargetSym."
        Set aDatTarSym = AnnoObject
        Set Anno = aDatTarSym.GetAnnotation
Case swSelDATUMTAGS
    Debug.Print "Selected annotation is a DatumTag."
    Set aDatumTag = AnnoObject
    Set Anno = aDatumTag.GetAnnotation
Case swSelDATUMPOINTS
    Debug.Print "Selected annotation is a DatumOrigin."
    Set aDatumOrigin = AnnoObject
    Set Anno = aDatumOrigin.GetAnnotation
Case swSelCUSTOMSYMBOLS
    Debug.Print "Selected annotation is a CustomSymbol."
    Set aCustomSymbol = AnnoObject
    Set Anno = aCustomSymbol.GetAnnotation
Case swSelCTHREADS
    Debug.Print "Selected annotation is a CThread."
    Set aCThread = AnnoObject
    Set Anno = aCThread.GetAnnotation
Case swSelCENTERMARKSYMS
    Debug.Print "Selected annotation is a CenterMark."
    Set aCenterMark = AnnoObject
    Set Anno = aCenterMark.GetAnnotation
Case swSelCENTERLINES
    Debug.Print "Selected annotation is a CenterLine."
    Set aCenterLine = AnnoObject
    Set Anno = aCenterLine.GetAnnotation
End Select

```

```

Debug.Print "Annotation: " & Anno.GetName

```

```

End Sub

```

Get Annotation Object Example (VB.NET)

This example shows how to obtain the annotation object for the selected annotation symbol.

```

'-----
' Preconditions:
' 1. Open a model document that contains an annotation symbol.
' 2. Select the annotation symbol.
'
' Postconditions: The annotation object for the selected symbol is obtained.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Dim Part As ModelDoc2
    Dim SelMgr As SelectionMgr
    Dim AnnoObject As Object
    Dim aWeldSymbol As WeldSymbol
    Dim aWeldBead As WeldBead
    Dim aView As View
    Dim aTableAnno As TableAnnotation
    Dim anSFSymbol As SFSymbol
    Dim aRevisionCloud As RevisionCloud
    Dim aNote As Note
    Dim aLeader As MultiJogLeader
    Dim aGtol As Gtol
    Dim aDowelSym As DowelSymbol
    Dim aDispDim As DisplayDimension
    Dim aDatumTargetSym As DatumTargetSym
    Dim aDatumTag As DatumTag
    Dim aDatumOrigin As DatumOrigin
    Dim aCustomSymbol As CustomSymbol
    Dim aCenterMark As CenterMark
    Dim aCenterLine As CenterLine
    Dim aCThread As CThread
    Dim Anno As Annotation

    Sub main()

        Part = swApp.ActiveDoc
        SelMgr = Part.SelectionManager
        AnnoObject = SelMgr.GetSelectedObject6(1, -1)

        Select Case SelMgr.GetSelectedObjectType3(1, -1)
            Case swSelectType_e.swSelWELDS
                Debug.Print("Selected annotation is a WeldSymbol.")
                aWeldSymbol = AnnoObject
                Anno = aWeldSymbol.GetAnnotation
            Case swSelectType_e.swSelWELDBEADS
                Debug.Print("Selected annotation is a WeldBead.")
                aWeldBead = AnnoObject
                Anno = aWeldBead.GetAnnotation
            Case swSelectType_e.swSelDRAWINGVIEWS
                Debug.Print("Selected annotation is a View.")
                aView = AnnoObject
                Anno = aView.GetAnnotation
            Case swSelectType_e.swSelANNOTATIONTABLES
                Debug.Print("Selected annotation is a TableAnnotation.")
                aTableAnno = AnnoObject
                Anno = aTableAnno.GetAnnotation
            Case swSelectType_e.swSelSFSYMBOLS
                Debug.Print("Selected annotation is an SFSymbol.")
                anSFSymbol = AnnoObject
                Anno = anSFSymbol.GetAnnotation
            Case swSelectType_e.swSelREVISIONCLOUDS
                Debug.Print("Selected annotation is a RevisionCloud.")
                aRevisionCloud = AnnoObject
                Anno = aRevisionCloud.GetAnnotation
            Case swSelectType_e.swSelNOTES
                Debug.Print("Selected annotation is a Note.")
                aNote = AnnoObject
                Anno = aNote.GetAnnotation
            Case swSelectType_e.swSelLEADERS
                Debug.Print("Selected annotation is a Leader.")
                aLeader = AnnoObject
                Anno = aLeader.GetAnnotation
            Case swSelectType_e.swSelGTOLS
                Debug.Print("Selected annotation is a Gtol.")
                aGtol = AnnoObject
                Anno = aGtol.GetAnnotation
            Case swSelectType_e.swSelDOWELSYMS
                Debug.Print("Selected annotation is a DowelSymbol.")
                aDowelSym = AnnoObject

```

```

        Anno = aDowelSym.GetAnnotation
    Case swSelectType_e.swSelDIMENSIONS
        Debug.Print("Selected annotation is a DisplayDimension.")
        aDispDim = AnnoObject
        Anno = aDispDim.GetAnnotation
    Case swSelectType_e.swSelDTMTARGS
        Debug.Print("Selected annotation is a DatumTargetSym.")
        aDatTarSym = AnnoObject
        Anno = aDatTarSym.GetAnnotation
    Case swSelectType_e.swSelDATUMTAGS
        Debug.Print("Selected annotation is a DatumTag.")
        aDatumTag = AnnoObject
        Anno = aDatumTag.GetAnnotation
    Case swSelectType_e.swSelDATUMPOINTS
        Debug.Print("Selected annotation is a DatumOrigin.")
        aDatumOrigin = AnnoObject
        Anno = aDatumOrigin.GetAnnotation
    Case swSelectType_e.swSelCUSTOMSYMBOLS
        Debug.Print("Selected annotation is a CustomSymbol.")
        aCustomSymbol = AnnoObject
        Anno = aCustomSymbol.GetAnnotation
    Case swSelectType_e.swSelCTHREADS
        Debug.Print("Selected annotation is a CThread.")
        aCThread = AnnoObject
        Anno = aCThread.GetAnnotation
    Case swSelectType_e.swSelCENTERMARKSYMS
        Debug.Print("Selected annotation is a CenterMark.")
        aCenterMark = AnnoObject
        Anno = aCenterMark.GetAnnotation
    Case swSelectType_e.swSelCENTERLINES
        Debug.Print("Selected annotation is a CenterLine.")
        aCenterLine = AnnoObject
        Anno = aCenterLine.GetAnnotation
End Select

```

```

        Debug.Print("Annotation: " & Anno.GetName)

```

```

End Sub

```

```

Public swApp As SldWorks

```

```

End Class

```

Get Annotations Arrays Example (VBA)

Methods added to the IView interface in SOLIDWORKS API 2009 SP1 can return arrays of each annotation type in a drawing view. This example shows how to call these methods.

```
'-----
' Preconditions: Verify that the specified path and template file exist.
'
' Postconditions:
' 1. Creates a part with annotations.
' 2. Click Save.
' 3. Creates a drawing with third-angle views of the part
'    and annotations.
' 4. Iterates through each annotation array and pops up message boxes
'    containing information about each annotation in the drawing views.
' 5. Click OK to close each message box.
'-----
Option Explicit

Const filedir As String = "C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\"
Const TemplateName As String = "C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\Drawing.drwdot"
Const TemplateSize As Long = swDwgTemplateBsize
Const PaperSize As Long = swDwgPaperBsize
Const ScaleNum As Double = 1#
Const ScaleDenom As Double = 2#
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatMgr As SldWorks.FeatureManager
Dim Response As Integer
Dim ThirdAngle As Boolean
Dim swDraw As SldWorks.DrawingDoc
Dim swSheet As SldWorks.Sheet
Dim i As Long
Dim j As Long
Dim retval As Boolean
Dim swView As SldWorks.View
Dim count As Long
Dim Annotations As Variant
Dim msg As String

Sub main()

    'Create a part
    Build_Part

    'Create a drawing of the part
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDraw = swApp.NewDocument(TemplateName, PaperSize, 0#, 0#)

    ThirdAngle = False
    swDraw.ActivateSheet "Sheet1"
    Set swSheet = swDraw.GetCurrentSheet
    swSheet.SetSize PaperSize, 0#, 0#
    swSheet.SetScale ScaleNum, ScaleDenom, True, True

    ' Create third-angle drawing views
    ' Pop up Save As dialog
    ' Click Save in the dialog
    retval = swDraw.Create3rdAngleViews2(swModel.GetPathName)

    Dim LocX As Double
    Dim LocY As Double
    LocX = 0.2635088599471
    LocY = 0.1934578136726

    ' Create isometric drawing view
    swDraw.CreateDrawViewFromModelView3 swModel.GetPathName, "*Isometric", LocX, LocY, 0
    swDraw.ViewDisplayShaded

    ' Insert display dimension annotations
    ' from the current model
    swDraw.InsertModelAnnotations3 0, swInsertDimensionsMarkedForDrawing, True, True, False, True

    ' Insert datum target symbol annotations
    ' from the current model
    swDraw.InsertModelAnnotations3 0, swInsertDatumTargets, True, True, False, True

    ' Insert geometric tolerance annotations
    ' from the current model
    swDraw.InsertModelAnnotations3 0, swInsertGTols, True, True, False, True

    ' Insert surface finish symbol annotations
    ' from the current model
    swDraw.InsertModelAnnotations3 0, swInsertSFSymbols, True, True, False, True

    ' Insert datum tag annotations from the current model
    swDraw.InsertModelAnnotations3 0, swInsertDatums, True, True, False, True

    swDraw.ForceRebuild3 True

    ' Iterate through all the views on the
    ' drawing to find display dimensions
    Set swView = swDraw.GetFirstView
    Do While Not swView Is Nothing
        count = swView.GetDisplayDimensionCount
        Dim DisplayDimension As SldWorks.DisplayDimension
```

```

Dim swDim As SldWorks.Dimension
' Iterate through all the display dimension
' annotations in each drawing view that has them
If count > 0 Then
    Annotations = swView.GetDisplayDimensions
    For j = 0 To UBound(Annotations)
        Set DisplayDimension = Annotations(j)
        Set swDim = DisplayDimension.GetDimension
        ' For each annotation in each drawing view,
        ' pop up a message box with the
        ' annotation name and corresponding dimension
        msg = "Display dimension found: " & swView.GetName2 & ":" & DisplayDimension.GetAnnotation.GetName & " = " & swDim.GetSystemValue2("")
        & " meters"
        swApp.SendMsgToUser2 msg, swMbInformation, swMbOk
    Next j
End If
Set swView = swView.GetNextView
Loop

' Iterate through all the views on the drawing
' to find datum target symbols
Set swView = swDraw.GetFirstView
Do While Not swView Is Nothing
    count = swView.GetDatumTargetSymCount
    Dim dtsymbol As SldWorks.DatumTargetSym
    ' Iterate through all the datum target symbol
    ' annotations in each drawing view that has them
    If count > 0 Then
        Annotations = swView.GetDatumTargetSyms
        For j = 0 To UBound(Annotations)
            Set dtsymbol = Annotations(j)
            ' For each annotation in each drawing view,
            ' pop up a message box with the
            ' annotation name and name of each
            ' datum target symbol found
            msg = "Datum target symbol found: " & swView.GetName2 & ":" & dtsymbol.GetAnnotation.GetName
            swApp.SendMsgToUser2 msg, swMbInformation, swMbOk
        Next j
    End If
    Set swView = swView.GetNextView
Loop

' Iterate through all the views on the drawing
' to find surface finish symbols
Set swView = swDraw.GetFirstView
Do While Not swView Is Nothing
    count = swView.GetSFSymbolCount
    Dim sfsymbol As SldWorks.sfsymbol
    ' Iterate through all the surface
    ' finish symbol annotations in each
    ' drawing view that has them
    If count > 0 Then
        Annotations = swView.GetSFSymbols
        For j = 0 To UBound(Annotations)
            Set sfsymbol = Annotations(j)
            ' For each annotation in each drawing
            ' view, pop up a message box with the
            ' annotation name and name of each
            ' surface finish symbol found
            msg = "Surface finish symbol found: " & swView.GetName2 & ":" & sfsymbol.GetAnnotation.GetName
            swApp.SendMsgToUser2 msg, swMbInformation, swMbOk
        Next j
    End If
    Set swView = swView.GetNextView
Loop

' Iterate through all the views on the
' drawing to find datum tags
Set swView = swDraw.GetFirstView
Do While Not swView Is Nothing
    count = swView.GetDatumTagCount
    Dim dTag As SldWorks.DatumTag
    ' Iterate through all the datum tags
    ' in each drawing view that has them
    If count > 0 Then
        Annotations = swView.GetDatumTags
        For j = 0 To UBound(Annotations)
            Set dTag = Annotations(j)
            ' For each annotation in each drawing view,
            ' pop up a message box with the
            ' annotation name and name
            ' of each datum tag found
            msg = "Datum tag found: " & swView.GetName2 & ":" & dTag.GetAnnotation.GetName
            swApp.SendMsgToUser2 msg, swMbInformation, swMbOk
        Next j
    End If
    Set swView = swView.GetNextView
Loop

' Iterate through all the views on the
' drawing to find geometric tolerances
Set swView = swDraw.GetFirstView
Do While Not swView Is Nothing
    count = swView.GetGTolCount
    Dim gtol As SldWorks.gtol
    ' Iterate through all the geometric
    ' tolerance symbols in each drawing view
    ' that has them
    If count > 0 Then
        Annotations = swView.GetGTols
        For j = 0 To UBound(Annotations)
            Set gtol = Annotations(j)
            ' For each annotation in each drawing

```

```

        ' view, pop up a message box with the
        ' annotation name and name of each
        ' geometric tolerance found
        msg = "Geometric tolerance symbol found: " & swView.GetName2 & ":" & gtol.GetAnnotation.GetName
        swApp.SendMsgToUser2 msg, swMbInformation, swMbOk
    Next j
End If
Set swView = swView.GetNextView
Loop

' In a similar fashion:
' Get other annotations on the drawing, if they exist
' Iterate through all the views on the drawing
' Get the annotation count, and if greater than zero,
' get the annotation array
' Iterate on each array, and set an annotation object
' to each array member
' Annotations = swView.GetDowelSymbols
' Annotations = swView.GetMultiJogLeaders
' Annotations = swView.GetDatumOrigins
' Annotations = swView.GetCenterLines
' Annotations = swView.GetCThreads
' Annotations = swView.GetWeldSymbols
' Annotations = swView.GetWeldBeads
' Annotations = swView.GetTableAnnotations

swApp.SetUserPreferenceToggle swInputDimValOnCreate, True
swModel.SetUserPreferenceToggle swDisplayAnnotations, False

End Sub

Private Sub Build_Part()
' Build a part with these annotations:
' display dimensions, geometric tolerance
' symbol, surface finish symbol,
' datum tag symbol, and datum target symbol
Set swApp = Application.SldWorks
Set swModel = swApp.NewDocument(filedir + "Part.prt", 0, 0, 0)
swModel.SetUserPreferenceIntegerValue swUnitsLinear, swMETER
swModel.SetUserPreferenceDoubleValue swMaterialPropertyDensity, 7800
swModel.SetUserPreferenceStringValue swMaterialPropertyCrosshatchPattern, "ISO (Steel)"
swModel.SketchManager.InsertSketch False

Dim Height As Double
Dim Width As Double
Height = 0.05
Width = 0.05

swApp.SetUserPreferenceToggle swInputDimValOnCreate, False
swModel.SketchManager.CreateLine 0.01, 0.01, 0, 0.01, 0.01 + Height, 0

swModel.ViewZoomtofit2

' Add display dimension to line
swModel.AddDimension2 0, 0.01 + Height / 2, 0

' Add geometric tolerance to line
swModel.InsertGtol
swModel.SketchManager.CreateLine 0.01, 0.01, 0, 0.01 + Width, 0.01, 0

swModel.ViewZoomtofit2

' Add display dimension
swModel.AddDimension2 0.01 + Width / 2, 0, 0

' Add surface finish symbol to line
swModel.Extension.InsertSurfaceFinishSymbol3 swSFBasic, swSTRAIGHT, 0.01, 0.01, 0.01, swSFCircular, swDOT_ARROWHEAD, "", "", "", "", "", "", ""

swModel.SketchManager.CreateLine(0, 0, 0, 0.01, 0).ConstructionGeometry = True

swModel.ClearSelection2 True
swModel.ViewZoomtofit2

Dim Thick As Double
Thick = 0.05

Dim Depth As Double
Depth = 0.05
Set swFeatMgr = swModel.FeatureManager
swFeatMgr.FeatureExtrusionThin2 True, False, True, 0, 0, Depth, 0, False, False, False, False, 0, 0, False, False, False, False, Thick, 0,
0, 0, 0, False, False, False, False, swStartSketchPlane, 0, False

swModel.ViewZoomtofit2
swModel.SetUserPreferenceToggle swDisplayAnnotations, True
swModel.ShowNamedView2 "Isometric", 7
swModel.ViewZoomtofit2

' Add datum tag to line
retval = swModel.Extension.SelectByID("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, False, 0, Nothing, 0)
Dim dTag As SldWorks.DatumTag
Set dTag = swModel.InsertDatumTag2

' Add datum target symbol to line
retval = swModel.Extension.SelectByID("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, False, 0, Nothing, 0)
Dim myDatumTarget As Object
Set myDatumTarget = swModel.Extension.InsertDatumTargetSymbol2("", "", "", 0, False, 0.03, 0.03, "", "", True, 12, 0, False, True, True)

End Sub

```

Get Annotations Arrays Example (C#)

Methods added to the IView interface in SOLIDWORKS API 2009 SP1 can return arrays of each annotation type in a drawing view. This example shows how to call these methods.

```
//-----
// Preconditions: Verify that the specified path and template file exist.
//
// Postconditions:
// 1. Creates a part with annotations.
// 2. Click Save.
// 3. Creates a drawing with third-angle views of the part
// and annotations.
// 4. Iterates through each annotation array and pops up message boxes
// containing information about each annotation in the drawing views.
// 5. Click OK to close each message box.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
namespace GetAnnotationsArrays.csproj
{
    partial class SolidWorksMacro
    {
        const string filedir = "C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\";
        const string TemplateName = "C:\\ProgramData\\SOLIDWORKS\\SOLIDWORKS 2016\\templates\\drawing.drwdot";
        const int TemplateSize = (int)swDwgTemplates_e.swDwgTemplateBsize;
        const int PaperSize = (int)swDwgPaperSizes_e.swDwgPaperBsize;
        const double ScaleNum = 1.0;
        const double ScaleDenom = 2.0;
        ModelDoc2 swModel;
        FeatureManager swFeatMgr;
        DrawingDoc swDraw;
        Sheet swSheet;
        long j;
        bool retval;
        View swView;
        long count;
        string msg;

        public void Main()
        {
            // Create a part
            Build_Part();

            // Create a drawing of the part
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swApp.NewDocument(TemplateName, PaperSize, 0.0, 0.0);
            swDraw.ActivateSheet("Sheet1");
            swSheet = (Sheet)swDraw.GetCurrentSheet();
            swSheet.SetSize(PaperSize, 0.0, 0.0);
            swSheet.SetScale(ScaleNum, ScaleDenom, true, true);

            // Create 3rd-angle drawing views
            // Pop up Save As dialog
            // Click Save in the dialog
            retval = swDraw.Create3rdAngleViews2(swModel.GetPathName());
            double LocX = 0;
            double LocY = 0;
            LocX = 0.2635088599471;
            LocY = 0.1934578136726;
            // Create isometric drawing view
            swDraw.CreateDrawViewFromModelView3(swModel.GetPathName(), "Isometric", LocX, LocY, 0);
            swDraw.ViewDisplayShaded();
            // Insert display dimension annotations from the current model
            swDraw.InsertModelAnnotations3(0, (int)swInsertAnnotation_e.swInsertDimensionsMarkedForDrawing, true, true, false, true);
            // Insert datum target symbol annotations from the current model
            swDraw.InsertModelAnnotations3(0, (int)swInsertAnnotation_e.swInsertDatumTargets, true, true, false, true);
            // Insert geometric tolerance annotations from the current model
            swDraw.InsertModelAnnotations3(0, (int)swInsertAnnotation_e.swInsertGTols, true, true, false, true);
            // Insert surface finish symbol annotations from the current model
            swDraw.InsertModelAnnotations3(0, (int)swInsertAnnotation_e.swInsertSFSymbols, true, true, false, true);
            // Insert datum tag annotations from the current model
            swDraw.InsertModelAnnotations3(0, (int)swInsertAnnotation_e.swInsertDatums, true, true, false, true);
            // Insert dowel symbol on a selected arc or circle - not applicable to this model
            //swDraw.InsertDowelSymbol
            // Insert center line on a selected entity
            //swDraw.InsertCenterLine2
            // Insert cosmetic thread
            //swDraw.InsertThreadCallout
            // Insert weld symbol on the last edge selection
            //swDraw.InsertWeldSymbol
            // Insert weld bead
            // Insert table
            //swDraw.InsertTableAnnotation2

            swDraw.ForceRebuild();

            // Iterate through all the views on the drawing to find display dimensions
            swView = (View)swDraw.GetFirstView();
            while ((swView != null))
            {
                count = swView.GetDisplayDimensionCount();
                DisplayDimension DisplayDimension = default(DisplayDimension);
                Dimension swDim = default(Dimension);
                // Iterate through all the display dimension annotations in each drawing view that has them
                if (count > 0)
                {
                    Object[] Annotations = (Object[])swView.GetDisplayDimensions();
                    for (j = 0; j <= Annotations.GetUpperBound(0); j++)
                    {
                        DisplayDimension = (DisplayDimension)Annotations[j];
                        swDim = (Dimension)DisplayDimension.GetDimension();
                        // For each annotation in each drawing view, pop up a message box with the
                        // annotation name and corresponding dimension
                        msg = "Display dimension found: " + swView.GetName2() + ":" + ((Annotation)DisplayDimension.GetAnnotation()).GetName() + " = " + swDim.GetSystemValue2("") + " meters";
                        swApp.SendMsgToUser2(msg, (int)swMessageBoxIcon_e.swMbInformation, (int)swMessageBoxBtn_e.swMbOk);
                    }
                }
                swView = (View)swView.GetNextView();
            }
        }
    }
}
```

```

}
// Iterate through all the views on the drawing to find datum target symbols
swView = (View)swDraw.GetFirstView();
while ((swView != null))
{
    count = swView.GetDatumTargetSymCount();
    DatumTargetSym dtsymbol = default(DatumTargetSym);
    // Iterate through all the datum target symbol annotations in each drawing view that has them
    if (count > 0)
    {
        Object[] Annotations = (Object[])swView.GetDatumTargetSyms();
        for (j = 0; j <= Annotations.GetUpperBound(0); j++)
        {
            dtsymbol = (DatumTargetSym)Annotations[j];
            // For each annotation in each drawing view, pop up a message box with the
            // annotation name and name of each datum target symbol found
            msg = "Datum target symbol found: " + swView.GetName2() + ":" + ((Annotation)dtsymbol.GetAnnotation()).GetName();
            swApp.SendMsgToUser2(msg, (int)swMessageBoxIcon_e.swMbInformation, (int)swMessageBoxBtn_e.swMbOk);
        }
    }
    swView = (View)swView.GetNextView();
}
// Iterate through all the views on the drawing to find surface finish symbols
swView = (View)swDraw.GetFirstView();
while ((swView != null))
{
    count = swView.GetSFSymbolCount();
    SFSymbol sfsymbol = default(SFSymbol);
    // Iterate through all the surface finish symbol annotations in each drawing view that has them
    if (count > 0)
    {
        Object[] Annotations = (Object[])swView.GetSFSymbols();
        for (j = 0; j <= Annotations.GetUpperBound(0); j++)
        {
            sfsymbol = (SFSymbol)Annotations[j];
            // For each annotation in each drawing view, pop up a message box with the
            // annotation name and name of each surface finish symbol found
            msg = "Surface finish symbol found: " + swView.GetName2() + ":" + ((Annotation)sfsymbol.GetAnnotation()).GetName();
            swApp.SendMsgToUser2(msg, (int)swMessageBoxIcon_e.swMbInformation, (int)swMessageBoxBtn_e.swMbOk);
        }
    }
    swView = (View)swView.GetNextView();
}
// Iterate through all the views on the drawing to find datum tags
swView = (View)swDraw.GetFirstView();
while ((swView != null))
{
    count = (int)swView.GetDatumTagCount();
    DatumTag dTag = default(DatumTag);
    // Iterate through all the datum tags in each drawing view that has them
    if (count > 0)
    {
        Object[] Annotations = (Object[])swView.GetDatumTags();
        for (j = 0; j <= Annotations.GetUpperBound(0); j++)
        {
            dTag = (DatumTag)Annotations[j];
            // For each annotation in each drawing view, pop up a message box with the
            // annotation name and name of each datum tag found
            msg = "Datum tag found: " + swView.GetName2() + ":" + ((Annotation)dTag.GetAnnotation()).GetName();
            swApp.SendMsgToUser2(msg, (int)swMessageBoxIcon_e.swMbInformation, (int)swMessageBoxBtn_e.swMbOk);
        }
    }
    swView = (View)swView.GetNextView();
}
// Iterate through all the views on the drawing to find geometric tolerances
swView = (View)swDraw.GetFirstView();
while ((swView != null))
{
    count = (int)swView.GetGTolCount();
    Gtol gtol = default(Gtol);
    // Iterate through all the geometric tolerance symbols in each drawing view that has them
    if (count > 0)
    {
        Object[] Annotations = (Object[])swView.GetGTols();
        for (j = 0; j <= Annotations.GetUpperBound(0); j++)
        {
            gtol = (Gtol)Annotations[j];
            // For each annotation in each drawing view, pop up a message box with the
            // annotation name and name of each geometric tolerance found
            msg = "Geometric tolerance symbol found: " + swView.GetName2() + ":" + ((Annotation)gtol.GetAnnotation()).GetName();
            swApp.SendMsgToUser2(msg, (int)swMessageBoxIcon_e.swMbInformation, (int)swMessageBoxBtn_e.swMbOk);
        }
    }
    swView = (View)swView.GetNextView();
}
// In a similar fashion:
// Get other annotations on the drawing, if they exist
// Iterate through all the views on the drawing
// Get the annotation count, and if greater than zero, get the annotation array
// Iterate on each array, and Set an annotation object to each array member:
// Annotations = swView.GetDowelSymbols
// Annotations = swView.GetMultiJogLeaders
// Annotations = swView.GetDatumOrigins
// Annotations = swView.GetCenterLines
// Annotations = swView.GetCThreads
// Annotations = swView.GetWeldSymbols
// Annotations = swView.GetWeldBeads
// Annotations = swView.GetTableAnnotations

swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swInputDimValOnCreate, true);
swModel.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDisplayAnnotations, false);
}
private void Build_Part()
{
    // Builds a part with these annotations:
    // display dimensions, geometric tolerance symbol,
    // surface finish symbol, datum tag symbol, and datum target symbol
    swModel = (ModelDoc2)swApp.NewDocument(filedir + "part.prtdot", 0, 0.0, 0.0);
    swModel.SetUserPreferenceIntegerValue((int)swUserPreferenceIntegerValue_e.swUnitsLinear, (int)swLengthUnit_e.swMETER);
    swModel.SetUserPreferenceDoubleValue((int)swUserPreferenceDoubleValue_e.swMaterialPropertyDensity, 7800);
    swModel.SetUserPreferenceStringValue((int)swUserPreferenceStringValue_e.swMaterialPropertyCrosshatchPattern, "ISO (Steel)");
    swModel.SketchManager.InsertSketch(false);
    double Height = 0;
    double Width = 0;
    Height = 0.05;
    Width = 0.05;
    swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swInputDimValOnCreate, false);
    swModel.SketchManager.CreateLine(0.01, 0.01, 0, 0.01, 0.01 + Height, 0);
    swModel.ViewZoomToFit2();
    // Add display dimension to line

```

```

swModel.AddDimension2(0, 0.01 + Height / 2, 0);
// Add geometric tolerance to line
swModel.InsertGtol();
swModel.SketchManager.CreateLine(0.01, 0.01, 0, 0.01 + Width, 0.01, 0);
swModel.ViewZoomtofit2();
// Add display dimension
swModel.AddDimension2(0.01 + Width / 2, 0, 0);
// Add surface finish symbol to line
swModel.Extension.InsertSurfaceFinishSymbol3((int)swSFSymType_e.swSFBasic, (int)swLeaderStyle_e.swSTRAIGHT, 0.01, 0.01, 0.01, (int)swSFLaySym_
e.swSFCircular, (int)swArrowStyle_e.swDOT_ARROWHEAD, "", "", "",
"", "", "", "");
swModel.SketchManager.CreateLine(0, 0, 0, 0, 0.01, 0).ConstructionGeometry = true;
swModel.ClearSelection2(true);
swModel.ViewZoomtofit2();
double Thick = 0;
Thick = 0.05;
double Depth = 0;
Depth = 0.05;
swFeatMgr = swModel.FeatureManager;
swFeatMgr.FeatureExtrusionThin2(true, false, true, 0, 0, Depth, 0, false, false, false,
false, 0, 0, false, false, false, false, false, Thick, 0,
0, 0, 0, false, 0.0, false, false, (int)swStartConditions_e.swStartSketchPlane, 0.0, false
);
swModel.ViewZoomtofit2();
swModel.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swDisplayAnnotations, true);
swModel.ShowNamedView2("Isometric", 7);
swModel.ViewZoomtofit2();
// Add datum tag to line
retval = swModel.Extension.SelectByID2("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, false, 0, null, 0);
DatumTag dTag = default(DatumTag);
dTag = (DatumTag)swModel.InsertDatumTag2();
// Add datum target symbol to line
retval = swModel.Extension.SelectByID2("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, false, 0, null, 0);
object myDatumTarget = null;
myDatumTarget = swModel.Extension.InsertDatumTargetSymbol2("", "", "", 0, false, 0.03, 0.03, "", "", true,
12, 0, false, true, true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Annotations Arrays Example (VB.NET)

Methods added to the IView interface in SOLIDWORKS API 2009 SP1 can return arrays of each annotation type in a drawing view. This example shows how to call these methods.

```

-----
' Preconditions: Verify that the specified path and template file exist.
-----
Postconditions:
1. Creates a part with annotations.
2. Click Save.
3. Creates a drawing with third-angle views of the part
   and annotations.
4. Iterates through each annotation array and pops up message boxes
   containing information about each annotation in the drawing views.
5. Click OK to close each message box.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Const filedir As String = "C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\"
    Const TemplateName As String = "C:\ProgramData\SOLIDWORKS\SOLIDWORKS 2016\templates\drawing.drwdot"
    Const TemplateSize As Long = swDwgTemplates_e.swDwgTemplateBsize
    Const PaperSize As Long = swDwgPaperSizes_e.swDwgPaperBsize
    Const ScaleNum As Double = 1.0#
    Const ScaleDenom As Double = 2.0#
    Dim swModel As ModelDoc2
    Dim swFeatMgr As FeatureManager
    Dim Response As Integer
    Dim ThirdAngle As Boolean
    Dim swDraw As DrawingDoc
    Dim swSheet As Sheet
    Dim i As Long
    Dim j As Long
    Dim retval As Boolean
    Dim swView As View
    Dim count As Long
    Dim Annotations As Object
    Dim msg As String

    Public Sub main()
        'Create a part
        Build_Part()

        'Create a drawing of the part
        swModel = swApp.ActiveDoc
        swDraw = swApp.NewDocument(TemplateName, PaperSize, _
            0.0#, 0.0#)
        ThirdAngle = False
        swDraw.ActivateSheet("Sheet1")
        swSheet = swDraw.GetCurrentSheet
        swSheet.SetSize(PaperSize, 0.0#, 0.0#)
        swSheet.SetScale(ScaleNum, ScaleDenom, True, True)

        ' Create 3rd-angle drawing views
        ' Pop up Save As dialog
        ' Click Save in the dialog
        retval = swDraw.Create3rdAngleViews2(swModel.GetPathName)
        Dim LocX As Double
        Dim LocY As Double
        LocX = 0.2635088599471
        LocY = 0.1934578136726

        ' Create isometric drawing view
        swDraw.CreateDrawViewFromModelView3(swModel.GetPathName, "Isometric", LocX, LocY, 0)
        swDraw.ViewDisplayShaded()
        ' Insert display dimension annotations from the current model
        swDraw.InsertModelAnnotations3(0, swInsertAnnotation_e.swInsertDimensionsMarkedForDrawing, True, True, False, True)
        ' Insert datum target symbol annotations from the current model
        swDraw.InsertModelAnnotations3(0, swInsertAnnotation_e.swInsertDatumTargets, True, True, False, True)
        ' Insert geometric tolerance annotations from the current model
        swDraw.InsertModelAnnotations3(0, swInsertAnnotation_e.swInsertGTols, True, True, False, True)
        ' Insert surface finish symbol annotations from the current model
        swDraw.InsertModelAnnotations3(0, swInsertAnnotation_e.swInsertSFSymbols, True, True, False, True)
        ' Insert datum tag annotations from the current model
        swDraw.InsertModelAnnotations3(0, swInsertAnnotation_e.swInsertDatums, True, True, False, True)
        ' Insert dowel symbol on a selected arc or circle - not applicable to this model
        'swDraw.InsertDowelSymbol
        ' Insert center line on a selected entity
        'swDraw.InsertCenterLine2
        ' Insert cosmetic thread
        'swDraw.InsertThreadCallout
        ' Insert weld symbol on the last edge selection
        'swDraw.InsertWeldSymbol
        ' Insert weld bead
        ' Insert table
        'swDraw.InsertTableAnnotation2

        swDraw.ForceRebuild3(True)

        ' Iterate through all the views on the drawing to find display dimensions
        swView = swDraw.GetFirstView
        Do While Not swView.Is Nothing
            count = swView.GetDisplayDimensionCount
            Dim DisplayDimension As DisplayDimension
            Dim swDim As Dimension
            ' Iterate through all the display dimension annotations in each drawing view that has them
            If count > 0 Then
                Annotations = swView.GetDisplayDimensions
                For j = 0 To UBound(Annotations)
                    DisplayDimension = Annotations(j)
                    swDim = DisplayDimension.GetDimension
                    ' For each annotation in each drawing view, pop up a message box with the
                    ' annotation name and corresponding dimension
                    msg = "Display dimension found: " & swView.GetName2 & " : " & DisplayDimension.GetAnnotation.GetName & " = " & swDim.GetSystemValue2("")
                    & " meters"
                    swApp.SendMsgToUser2(msg, swMessageBoxIcon_e.swMbInformation, swMessageBoxBtn_e.swMbOk)
                Next j
            End If
        Next swView
    End Sub
End Class

```

```

End If
swView = swView.GetNextView
Loop
' Iterate through all the views on the drawing to find datum target symbols
swView = swDraw.GetFirstView
Do While Not swView Is Nothing
count = swView.GetDatumTargetSymCount
Dim dtsymbol As DatumTargetSym
' Iterate through all the datum target symbol annotations in each drawing view that has them
If count > 0 Then
Annotations = swView.GetDatumTargetSyms
For j = 0 To UBound(Annotations)
dtsymbol = Annotations(j)
' For each annotation in each drawing view, pop up a message box with the
' annotation name and name of each datum target symbol found
msg = "Datum target symbol found: " & swView.GetName2 & ":" & dtsymbol.GetAnnotation.GetName
swApp.SendMsgToUser2(msg, swMessageBoxIcon_e.swMbInformation, swMessageBoxBtn_e.swMbOk)
Next j
End If
swView = swView.GetNextView
Loop
' Iterate through all the views on the drawing to find surface finish symbols
swView = swDraw.GetFirstView
Do While Not swView Is Nothing
count = swView.GetSFSymbolCount
Dim sfsymbol As SFSymbol
' Iterate through all the surface finish symbol annotations in each drawing view that has them
If count > 0 Then
Annotations = swView.GetSFSymbols
For j = 0 To UBound(Annotations)
SFSymbol = Annotations(j)
' For each annotation in each drawing view, pop up a message box with the
' annotation name and name of each surface finish symbol found
msg = "Surface finish symbol found: " & swView.GetName2 & ":" & SFSymbol.GetAnnotation.GetName
swApp.SendMsgToUser2(msg, swMessageBoxIcon_e.swMbInformation, swMessageBoxBtn_e.swMbOk)
Next j
End If
swView = swView.GetNextView
Loop
' Iterate through all the views on the drawing to find datum tags
swView = swDraw.GetFirstView
Do While Not swView Is Nothing
count = swView.GetDatumTagCount
Dim dTag As DatumTag
' Iterate through all the datum tags in each drawing view that has them
If count > 0 Then
Annotations = swView.GetDatumTags
For j = 0 To UBound(Annotations)
dTag = Annotations(j)
' For each annotation in each drawing view, pop up a message box with the
' annotation name and name of each datum tag found
msg = "Datum tag found: " & swView.GetName2 & ":" & dTag.GetAnnotation.GetName
swApp.SendMsgToUser2(msg, swMessageBoxIcon_e.swMbInformation, swMessageBoxBtn_e.swMbOk)
Next j
End If
swView = swView.GetNextView
Loop
' Iterate through all the views on the drawing to find geometric tolerances
swView = swDraw.GetFirstView
Do While Not swView Is Nothing
count = swView.GetGtolCount
Dim gtol As Gtol
' Iterate through all the geometric tolerance symbols in each drawing view that has them
If count > 0 Then
Annotations = swView.GetGTols
For j = 0 To UBound(Annotations)
Gtol = Annotations(j)
' For each annotation in each drawing view, pop up a message box with the
' annotation name and name of each geometric tolerance found
msg = "Geometric tolerance symbol found: " & swView.GetName2 & ":" & Gtol.GetAnnotation.GetName
swApp.SendMsgToUser2(msg, swMessageBoxIcon_e.swMbInformation, swMessageBoxBtn_e.swMbOk)
Next j
End If
swView = swView.GetNextView
Loop
' In a similar fashion:
' Get other annotations on the drawing, if they exist
' Iterate through all the views on the drawing
' Get the annotation count, and if greater than zero, get the annotation array
' Iterate on each array, and set an annotation object to each array member:
Annotations = swView.GetDowelSymbols
Annotations = swView.GetMultiJogLeaders
Annotations = swView.GetDatumOrigins
Annotations = swView.GetCenterLines
Annotations = swView.GetCThreads
Annotations = swView.GetWeldSymbols
Annotations = swView.GetWeldBeads
Annotations = swView.GetTableAnnotations
swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swInputDimValOnCreate, True)
swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayAnnotations, False)

End Sub

Private Sub Build_Part()
' Builds a part with these annotations:
' display dimensions, geometric tolerance symbol, surface finish symbol,
' datum tag symbol, and datum target symbol
swModel = swApp.NewDocument(filedir & "part.prtdot", 0, 0.0#, 0.0#)
swModel.SetUserPreferenceIntegerValue(swUserPreferenceIntegerValue_e.swUnitsLinear, swLengthUnit_e.swMETER)
swModel.SetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swMaterialPropertyDensity, 7800)
swModel.SetUserPreferenceStringValue(swUserPreferenceStringValue_e.swMaterialPropertyCrosshatchPattern, "ISO (Steel)")
swModel.SketchManager.InsertSketch(False)
Dim Height As Double
Dim Width As Double
Height = 0.05
Width = 0.05
swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swInputDimValOnCreate, False)
swModel.SketchManager.CreateLine(0.01, 0.01, 0, 0.01, 0.01 + Height, 0)
swModel.ViewZoomToFit2()
' Add display dimension to line
swModel.AddDimension2(0, 0.01 + Height / 2, 0)
' Add geometric tolerance to line
swModel.InsertGtol()
swModel.SketchManager.CreateLine(0.01, 0.01, 0, 0.01 + Width, 0.01, 0)
swModel.ViewZoomToFit2()
' Add display dimension
swModel.AddDimension2(0.01 + Width / 2, 0, 0)
' Add surface finish symbol to line
swModel.Extension.InsertSurfaceFinishSymbol3(swSFSymType_e.swSFBasic, swLeaderStyle_e.swSTRAIGHT, 0.01, 0.01, 0.01, swSFLaySym_e.swSFCircular, swA
rrowStyle_e.swDOT_ARROWHEAD, "", "", "", "", "", "", "")

```

```

swModel.SketchManager.CreateLine(0, 0, 0, 0, 0.01, 0).ConstructionGeometry = True
swModel.ClearSelection2(True)
swModel.ViewZoomtofit2()
Dim Thick As Double
Thick = 0.05
Dim Depth As Double
Depth = 0.05
swFeatMgr = swModel.FeatureManager
swFeatMgr.FeatureExtrusionThin2(True, False, True, 0, 0, Depth, 0, False, False, False, False, 0, 0, False, False, False, False, False, Thick, 0,
0, 0, 0, False, False, False, False, swStartConditions_e.swStartSketchPlane, 0.0#, False)
swModel.ViewZoomtofit2()
swModel.SetUserPreferenceToggle(swUserPreferenceToggle_e.swDisplayAnnotations, True)
swModel.ShowNamedView2("Isometric", 7)
swModel.ViewZoomtofit2()
' Add datum tag to line
retval = swModel.Extension.SelectByID("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, False, 0, Nothing, 0)
Dim dTag As DatumTag
dTag = swModel.InsertDatumTag2
' Add datum target symbol to line
retval = swModel.Extension.SelectByID("", "EDGE", 0.06001738353251, -0.01284975502705, -0.05001738353241, False, 0, Nothing, 0)
Dim myDatumTarget As Object
myDatumTarget = swModel.Extension.InsertDatumTargetSymbol2("", "", "", 0, False, 0.03, 0.03, "", "", True, 12, 0, False, True, True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Annotations Example (C#)

This example shows how to get the annotations of a model.

```
//-----
// Preconditions:
// 1. Open a model with one or more annotations.
// 2. Open an Immediate window.
//
// Postconditions: All the annotations are printed in the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace GetAnnotations_CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            Annotation swAnnotation;
            int iAnnoCnt;
            Object[] arrAnnotation;
            int IAnnoType;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            iAnnoCnt = swModelDocExt.GetAnnotationCount();

            if (iAnnoCnt > 0)
            {
                arrAnnotation = (Object[])swModelDocExt.GetAnnotations();

                for (int i = 0; i <= arrAnnotation.Length - 1; i++)
                {
                    swAnnotation = (Annotation)arrAnnotation[i];

                    IAnnoType = swAnnotation.GetType();

                    switch (IAnnoType)
                    {
                        case 1:
                            Debug.Print("Annotation: Thread");
                            break;
                        case 2:
                            Debug.Print("Annotation: Datum Tag");
                            break;
                        case 3:
                            Debug.Print("Annotation: Datum Target Symbol");
                            break;
                        case 4:
                            Debug.Print("Annotation: Display Dimension");
                            break;
                        case 5:
                            Debug.Print("Annotation: Gtol");
                            break;
                        case 6:
                            Debug.Print("Annotation: Note");
                            break;
                        case 7:
                            Debug.Print("Annotation: SFS Symbol");
                            break;
                        case 8:
                            Debug.Print("Annotation: Weld Symbol");
                            break;
                        case 9:
                            Debug.Print("Annotation: Custom Symbol");
                            break;
                        case 10:
                            Debug.Print("Annotation: Dowel Symbol");
                            break;
                        case 11:
                            Debug.Print("Annotation: Leader");
                            break;
                        case 12:
                            Debug.Print("Annotation: Block");
                            break;
                        case 13:
                            Debug.Print("Annotation: Center Mark symbol");
                    }
                }
            }
        }
    }
}
```

```
        break;
    case 14:
        Debug.Print("Annotation: Table Annotation");
        break;
    case 15:
        Debug.Print("Annotation: Center Line ");
        break;
    case 16:
        Debug.Print("Annotation: Datum Origin");
        break;
    }
}
}
}
}
}

    public SldWorks swApp;
}
}
```

Get Annotations Example (VBA)

This example shows how to get the annotations of a model.

```
'-----
' Preconditions:
' 1. Open a model with one or more annotations.
' 2. Open an Immediate window.
'
' Postconditions: All the annotations are printed in the Immediate window.
'-----
```

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAnnotation As SldWorks.Annotation
Dim iAnnoCnt As Integer
Dim arrAnnotation As Variant
Dim i As Integer
Dim IAnnoType As Integer
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    iAnnoCnt = swModelDocExt.GetAnnotationCount()

    If iAnnoCnt > 0 Then

        arrAnnotation = swModelDocExt.GetAnnotations()

        For i = LBound(arrAnnotation) To UBound(arrAnnotation)

            Set swAnnotation = arrAnnotation(i)

            IAnnoType = swAnnotation.GetType()

            Select Case IAnnoType

                Case 1
                    Debug.Print "Annotation: Thread"

                Case 2
                    Debug.Print "Annotation: Datum Tag"

                Case 3
                    Debug.Print "Annotation: Datum Target Symbol"

                Case 4
                    Debug.Print "Annotation: Display Dimension"

                Case 5
                    Debug.Print "Annotation: Gtol"

                Case 6
                    Debug.Print "Annotation: Note"

                Case 7
                    Debug.Print "Annotation: SFS Symbol"

                Case 8
                    Debug.Print "Annotation: Weld Symbol"

                Case 9
                    Debug.Print "Annotation: Custom Symbol"

                Case 10
                    Debug.Print "Annotation: Dowel Symbol"

                Case 11
                    Debug.Print "Annotation: Leader"

                Case 12
                    Debug.Print "Annotation: Block"
```

```
Case 13
    Debug.Print "Annotation: Center Mark symbol"
```

```
Case 14
    Debug.Print "Annotation: Table Annotation"
```

```
Case 15
    Debug.Print "Annotation: Center Line"
```

```
Case 16
    Debug.Print "Annotation: Datum Origin"
```

```
End Select
```

```
Next
```

```
End If
```

```
End Sub
```

Get Annotations Example (VB.NET)

This example shows how to get the annotations of a model.

```

'-----
' Preconditions:
' 1. Open a model with one or more annotations.
' 2. Open an Immediate window.
'
' Postconditions: All the annotations are printed in the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAnnotation As Annotation
        Dim iAnnoCnt As Integer
        Dim arrAnnotation As Object
        Dim i As Integer
        Dim IAnnoType As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        iAnnoCnt = swModelDocExt.GetAnnotationCount()

        If iAnnoCnt > 0 Then
            arrAnnotation = swModelDocExt.GetAnnotations()

            For i = LBound(arrAnnotation) To UBound(arrAnnotation)

                swAnnotation = arrAnnotation(i)

                IAnnoType = swAnnotation.GetType()

                Select Case IAnnoType

                    Case 1
                        Debug.Print("Annotation: Thread")

                    Case 2
                        Debug.Print("Annotation: Datum Tag")

                    Case 3
                        Debug.Print("Annotation: Datum Target Symbol")

                    Case 4
                        Debug.Print("Annotation: Display Dimension")

                    Case 5
                        Debug.Print("Annotation: Gtol")

                    Case 6
                        Debug.Print("Annotation: Note")

                    Case 7
                        Debug.Print("Annotation: SFS Symbol")

                    Case 8
                        Debug.Print("Annotation: Weld Symbol")

                    Case 9
                        Debug.Print("Annotation: Custom Symbol")

                    Case 10
                        Debug.Print("Annotation: Dowel Symbol")

                    Case 11
                        Debug.Print("Annotation: Leader")

                    Case 12
                        Debug.Print("Annotation: Block")

                    Case 13
                        Debug.Print("Annotation: Center Mark symbol")

                    Case 14
                        Debug.Print("Annotation: Table Annotation")

                    Case 15

```

```
        Debug.Print("Annotation: Center Line ")
    Case 16
        Debug.Print("Annotation: Datum Origin")
    End Select
Next
End If
End Sub

Public swApp As SldWorks

End Class
```

Get Appearance File Name Example (C#)

This example shows how to get the file name of the first appearance applied to a model.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. Verify that the specified materials file exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified model document.
// 2. Applies the specified appearance to the selected face.
// 3. Gets the material IDs.
// 4. Gets the file name of the first appearance applied to the model.
// 5. Examine the graphics area and the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace FileNameCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Face2 swFace = default(Face2);
            Configuration swConfiguration = default(Configuration);
            RenderMaterial swRenderMaterial = default(RenderMaterial);
            int nbrRenderMaterials = 0;
            string fileName = null;
            int warnings = 0;
            int errors = 0;
            bool status = false;
            string materialName = null;
            string[] displayStateNames = null;
            int materialID1 = 0;
            int materialID2 = 0;

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\cover_plate.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            //Create material for the appearance
            materialName = "C:\\Program Files\\SolidWorks Corp\\SolidWorks\\data\\graphics\\Materials\\plastic\\low gloss\\blue low gloss plastic.p2m";
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swRenderMaterial = (RenderMaterial)swModelDocExt.CreateRenderMaterial(materialName);

            //Verify that RealView Graphics is set
            Debug.Print("RealView Graphics set? " + swModelDocExt.ViewDisplayRealView);

            //Select a face and add an appearance
            status = swModelDocExt.SelectByID2("", "FACE", 0.0417924256550464, 0.0796803314056547, 0, false, 0, null, 0);
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelectionMgr.GetSelectedObject6(1, -1);
            status = swRenderMaterial.AddEntity(swFace);
            Debug.Print("Appearance added to selected face? " + status);

            swModel.ClearSelection2(true);

            //Get the names of display states
            //Add the appearance to all display states
            swConfiguration = (Configuration)swModel.GetActiveConfiguration();
            displayStateNames = (string[])swConfiguration.GetDisplayStates();
            status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, (int)swDisplayStateOpts_e.swAllDisplayState, displayStateNames,
out materialID1, out materialID2);
            Debug.Print("Material IDs returned by IModelDocExtension::AddDisplayStateSpecificRenderMaterial: ");
            Debug.Print(" ID1: " + materialID1);
            Debug.Print(" ID2: " + materialID2);

            //Get the number of appearances
            nbrRenderMaterials = swModelDocExt.GetRenderMaterialsCount2((int)swDisplayStateOpts_e.swAllDisplayState, displayStateNames);

            //If one or more appearances are applied to the model,
            //then get the file name of the first appearance applied
            if (nbrRenderMaterials > 0)
            {
                swRenderMaterial.GetMaterialIds(out materialID1, out materialID2);
                Debug.Print("Material IDs returned by IModelDocExtension::GetMaterialIds: ");
                Debug.Print(" ID1: " + materialID1);
                Debug.Print(" ID2: " + materialID2);
                Debug.Print("First appearance's file name: " + swRenderMaterial.FileName);
            }
            else
            {
                Debug.Print("No appearances applied to this model.");
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Get Appearance File Name Example (VBA)

This example shows how to get the file name of the first appearance applied to a model.

```
'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. Verify that the specified materials file exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified model document.
' 2. Applies the specified appearance to the selected face.
' 3. Gets the material IDs.
' 4. Gets the file name of the first appearance applied to the model.
' 5. Examine the graphics area and the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim swConfiguration As SldWorks.Configuration
Dim swRenderMaterial As SldWorks.RenderMaterial
Dim nbrRenderMaterials As Long
Dim fileName As String
Dim warnings As Long
Dim errors As Long
Dim status As Boolean
Dim materialName As String
Dim displayStateNames() As String
Dim materialID1 As Long
Dim materialID2 As Long

Sub main()

    Set swApp = Application.SldWorks

    'Open model document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cover_plate.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    'Create material for the appearance
    materialName = "C:\Program Files\SolidWorks Corp\SolidWorks\data\graphics\Materials\plastic\low gloss\blue low gloss plastic.p2m"
    Set swModelDocExt = swModel.Extension
    Set swRenderMaterial = swModelDocExt.CreateRenderMaterial(materialName)

    'Verify that RealView Graphics is set
    Debug.Print "RealView Graphics set? " & swModelDocExt.ViewDisplayRealView

    'Select a face and add an appearance
    status = swModelDocExt.SelectByID2("", "FACE", 4.17924256550464E-02, 7.96803314056547E-02, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFace = swSelectionMgr.GetSelectedObject6(1, -1)
    status = swRenderMaterial.AddEntity(swFace)
    Debug.Print "Appearance added to selected face? " & status

    swModel.ClearSelection2 True

    'Get the names of display states
    'Add the appearance to all display states
    Set swConfiguration = swModel.GetActiveConfiguration
    displayStateNames = swConfiguration.GetDisplayStates
    status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, swDisplayStateOpts_e.swAllDisplayState, displayStateNames, materialID1, materialID2)
    Debug.Print "Material IDs returned by IModelDocExtension::AddDisplayStateSpecificRenderMaterial: "
    Debug.Print "   ID1: " & materialID1
    Debug.Print "   ID2: " & materialID2

    'Get the number of appearances
    nbrRenderMaterials = swModelDocExt.GetRenderMaterialsCount2(swDisplayStateOpts_e.swAllDisplayState, displayStateNames)

    'If one or more appearances are applied to the model,
    'then get the file name of the first appearance applied
    If nbrRenderMaterials > 0 Then
        swRenderMaterial.GetMaterialIds materialID1, materialID2
        Debug.Print "Material IDs returned by IModelDocExtension::GetMaterialIds: "
        Debug.Print "   ID1: " & materialID1
        Debug.Print "   ID2: " & materialID2
        Debug.Print "First appearance's file name: " & swRenderMaterial.fileName
    Else
        Debug.Print "No appearances applied to this model."
    End If
End Sub
```

Get Appearance File Name Example (VB.NET)

This example shows how to get the file name of the first appearance applied to a model.

```

-----
Preconditions:
1. Verify that the specified model document exists.
2. Verify that the specified materials file exists.
3. Open the Immediate window.
Postconditions:
1. Opens the specified model document.
2. Applies the specified appearance to the selected face.
3. Gets the material IDs.
4. Gets the file name of the first appearance applied to the model.
5. Examine the graphics area and the Immediate window.
NOTE: Because the model is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim swFace As Face2
        Dim swConfiguration As Configuration
        Dim swRenderMaterial As RenderMaterial
        Dim nbrRenderMaterials As Integer
        Dim fileName As String
        Dim warnings As Integer
        Dim errors As Integer
        Dim status As Boolean
        Dim materialName As String
        Dim displayStateNames() As String
        Dim materialID1 As Integer
        Dim materialID2 As Integer

        'Open model document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cover_plate.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        'Create material for the appearance
        materialName = "C:\Program Files\SolidWorks Corp\SolidWorks\data\graphics\Materials\plastic\low gloss\blue low gloss plastic.p2m"
        swModelDocExt = swModel.Extension
        swRenderMaterial = swModelDocExt.CreateRenderMaterial(materialName)

        'Verify that RealView Graphics is set
        Debug.Print("RealView Graphics set? " & swModelDocExt.ViewDisplayRealView)

        'Select a face and add an appearance
        status = swModelDocExt.SelectByID("", "FACE", 0.0417924256550464, 0.0796803314056547, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swFace = swSelectionMgr.GetSelectedObject6(1, -1)
        status = swRenderMaterial.AddEntity(swFace)
        Debug.Print("Appearance added to selected face? " & status)

        swModel.ClearSelection2(True)

        'Get the names of display states
        'Add the appearance to all display states
        swConfiguration = swModel.GetActiveConfiguration
        displayStateNames = swConfiguration.GetDisplayStates
        status = swModelDocExt.AddDisplayStateSpecificRenderMaterial(swRenderMaterial, swDisplayStateOpts_e.swAllDisplayState, displayStateNames, material
ID1, materialID2)
        Debug.Print("Material IDs returned by IModelDocExtension::AddDisplayStateSpecificRenderMaterial: ")
        Debug.Print(" ID1: " & materialID1)
        Debug.Print(" ID2: " & materialID2)

        'Get the number of appearances
        nbrRenderMaterials = swModelDocExt.GetRenderMaterialsCount2(swDisplayStateOpts_e.swAllDisplayState, displayStateNames)

        'If one or more appearances are applied to the model,
        'then get the file name of the first appearance applied
        If nbrRenderMaterials > 0 Then
            swRenderMaterial.GetMaterialIds(materialID1, materialID2)
            Debug.Print("Material IDs returned by IModelDocExtension::GetMaterialIds: ")
            Debug.Print(" ID1: " & materialID1)
            Debug.Print(" ID2: " & materialID2)
            Debug.Print("First appearance's file name: " & swRenderMaterial.FileName)
        Else
            Debug.Print("No appearances applied to this model.")
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Arcs in Sketch Example (VBA)

This example shows how to get the arcs in a sketch.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cstick.sldprt.
' 2. Expand Revolve1 in the FeatureManager design tree
'    and select Sketch1.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets all of the arcs in the sketch and their
'    information.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Enum swLineTypes_e
    swLF_VISIBLE = 0
    swLF_HIDDEN = 1
    swLF_SKETCH = 2
    swLF_DETAIL = 3
    swLF_SECTION = 4
    swLF_DIMENSION = 5
    swLF_CENTER = 6
    swLF_HATCH = 7
    swLF_TANGENT = 8
End Enum

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFeat As SldWorks.Feature
    Dim swSketch As SldWorks.Sketch
    Dim NumArcs As Long
    Dim vArcs As Variant
    Dim i As Variant
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swPart = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swFeat = swSelMgr.GetSelectedObject5(1)
    Set swSketch = swFeat.GetSpecificFeature
    NumArcs = swSketch.GetArcCount
    Debug.Print "Feature = " & swFeat.GetTypeName
    Debug.Print "  NumArcs = " & NumArcs
    Debug.Print ""

    vArcs = swSketch.GetArcs2
    If IsEmpty(vArcs) Then Exit Sub

    Debug.Assert UBound(vArcs) + 1 = 16 * NumArcs
    For i = 0 To NumArcs - 1
        Debug.Print "  Arc(" & i & ") "
        Debug.Print "    colour      = " & vArcs(16 * i + 0)
        Debug.Print "    type        = " & vArcs(16 * i + 1)
        Debug.Print "    font        = " & vArcs(16 * i + 2)
        Debug.Print "    width       = " & vArcs(16 * i + 3)
        Debug.Print "    layerID     = " & vArcs(16 * i + 4)
        Debug.Print "    layer override = " & vArcs(16 * i + 5)
        Debug.Print "    start       = (" & vArcs(16 * i + 6) * 1000# & ", " & vArcs(16 * i + 7) * 1000# & ", " & vArcs(16 * i + 8) * 1000# & ") mm"
        Debug.Print "    end         = (" & vArcs(16 * i + 9) * 1000# & ", " & vArcs(16 * i + 10) * 1000# & ", " & vArcs(16 * i + 11) * 1000# & ")
mm"
        Debug.Print "    ctr         = (" & vArcs(16 * i + 12) * 1000# & ", " & vArcs(16 * i + 13) * 1000# & ", " & vArcs(16 * i + 14) * 1000# & ")
mm"
        Debug.Print "    RotDir      = " & vArcs(16 * i + 15)

    Next i
End Sub
```